

models_comparison(24-01-2026)

January 24, 2026

```
[21]: import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

PRED_DIR = "../outputs/models/predictions/"
PLOT_DIR = "../outputs/models/plots/comparisons/"
os.makedirs(PLOT_DIR, exist_ok=True)

MODELS = {
    "LR": "lr",
    "RF": "rf",
    "LSTM": "lstm",
    "GRU": "gru",
    "CNN-LSTM": "cnn_lstm",
    "Transformer": "transformer",
    "GNN": "gnn",
}

sns.set_style("whitegrid")

def load_preds(model_key, metric, split="test"):
    path = os.path.join(
        PRED_DIR, f"{model_key}_{metric}_{split}_predictions.csv"
    )
    df = pd.read_csv(path)
    return df["y_true"], df["y_pred"]

def plot_model_comparison_pred_vs_actual(
    metric,
    split="test",
    showImage=False
):
    plt.figure(figsize=(8, 8))
```

```

all_true = []
all_pred = []

# Plot each model
for name, key in MODELS.items():
    y_true, y_pred = load_preds(key, metric, split)

    sns.scatterplot(
        x=y_true,
        y=y_pred,
        alpha=0.55,
        label=name,
        s=40
    )

    all_true.append(y_true)
    all_pred.append(y_pred)

# Global diagonal
y_min = min([s.min() for s in all_true + all_pred])
y_max = max([s.max() for s in all_true + all_pred])

plt.plot(
    [y_min, y_max],
    [y_min, y_max],
    "k--",
    linewidth=2,
    label="Ideal"
)

plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title(f"{metric} - Predicted vs Actual (Model Comparison, {split}.
↪capitalize()))")
plt.legend()
plt.tight_layout()

out = os.path.join(
    PLOT_DIR, f"compare_{metric.lower()}_{split}_pred_vs_actual.png"
)
plt.savefig(out, dpi=300)
print(f"Saved → {out}")

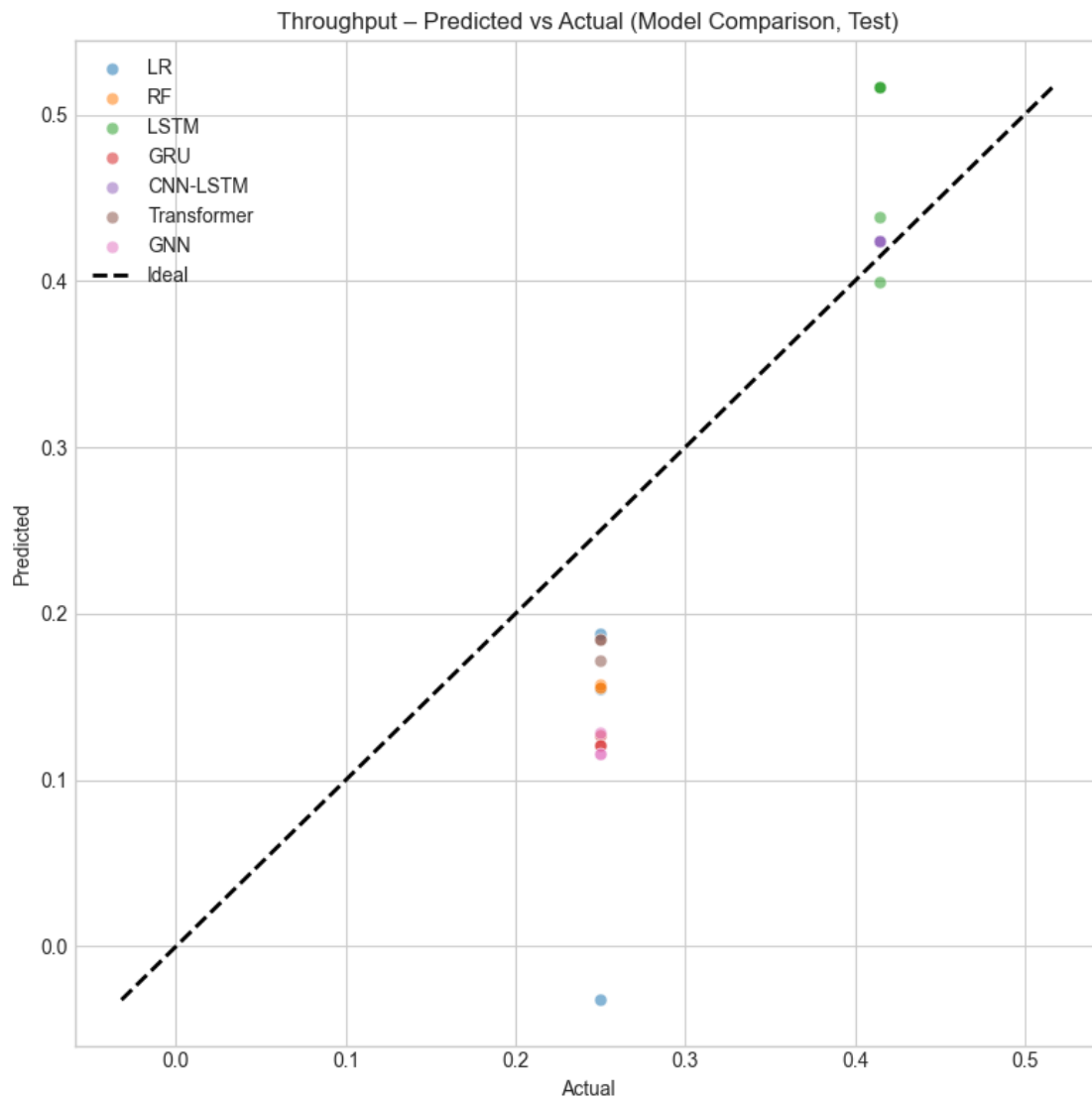
if showImage:
    plt.show()
else:
    plt.close()

```

```
[22]: plot_model_comparison_pred_vs_actual(
        metric="Throughput",
        split="test",
        showImage=True
    )
```

Saved →

../outputs/models/plots/comparisons/compare_throughput_test_pred_vs_actual.png

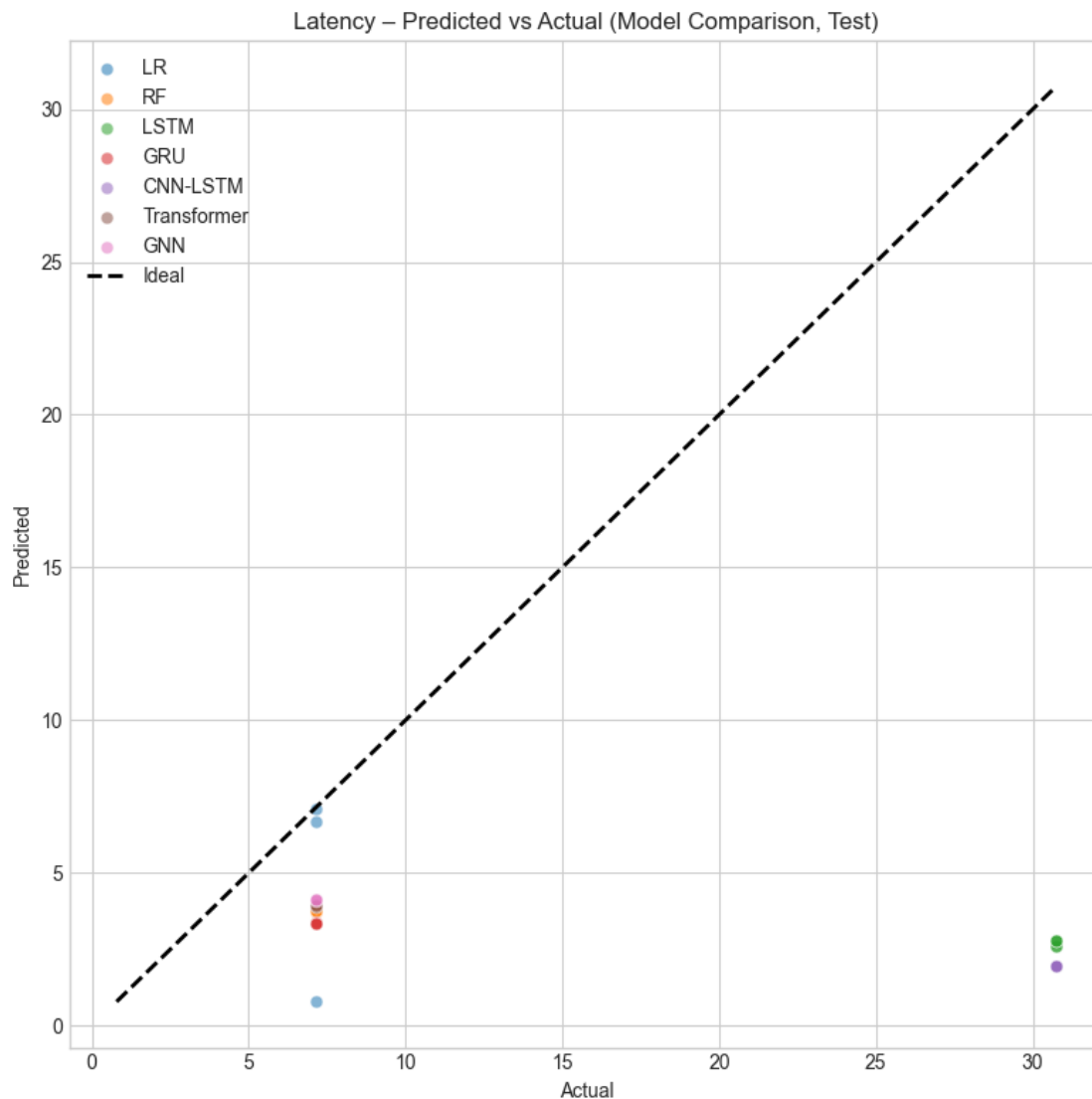


```
[23]: plot_model_comparison_pred_vs_actual(
        metric="Latency",
        split="test",
        showImage=True
    )
```

```
)
```

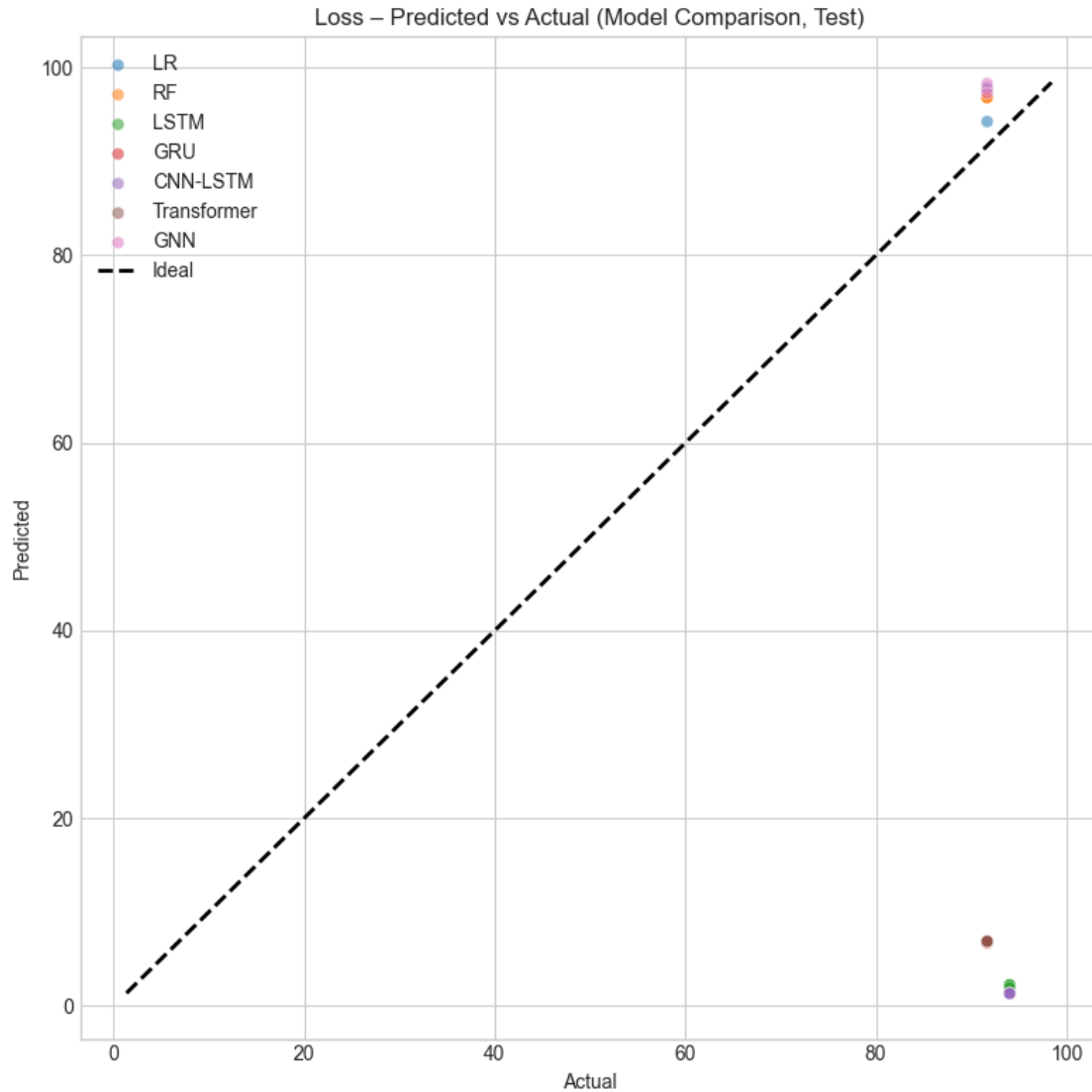
Saved →

../outputs/models/plots/comparisons/compare_latency_test_pred_vs_actual.png



```
[24]: plot_model_comparison_pred_vs_actual(  
    metric="Loss",  
    split="test",  
    showImage=True  
)
```

Saved → ../outputs/models/plots/comparisons/compare_loss_test_pred_vs_actual.png



```
[25]: import os
import pandas as pd
import matplotlib.pyplot as plt

PRED_DIR = "../outputs/models/predictions/"
PLOT_DIR = "../outputs/models/plots/comparisons/"
os.makedirs(PLOT_DIR, exist_ok=True)

# Model → (file_key, color, linestyle)
MODELS = {
    "LR": ("lr", "#1f77b4", ":"), # blue, dotted
    "RF": ("rf", "#ff7f0e", "-."), # orange, dash-dot
    "LSTM": ("lstm", "#2ca02c", "--"), # green, dashed

```

```

    "GRU":          ("gru",          "#d62728", "--"),    # red, dashed
    "CNN-LSTM":     ("cnn_lstm",     "#9467bd", "-"),      # purple, solid
    "Transformer":  ("transformer",  "#8c564b", (0, (5, 2))), # brown, long-dash
    "GNN":          ("gnn",          "#e377c2", (0, (3, 1, 1, 1))), # pink, 
↪ custom
}

plt.style.use("seaborn-v0_8-whitegrid")

def load_preds(model_key, metric, split="test"):
    path = os.path.join(
        PRED_DIR, f"{model_key}_{metric}_{split}_predictions.csv"
    )
    df = pd.read_csv(path)
    return df["y_true"], df["y_pred"]

def plot_model_comparison_time_series(
    metric,
    split="test",
    max_points=300,
    showImage=False
):
    plt.figure(figsize=(13, 6))

    # -----
    # Ground truth
    # -----
    first_key = next(iter(MODELS.values()))[0]
    y_true, _ = load_preds(first_key, metric, split)
    y_true = y_true.iloc[:max_points]

    plt.plot(
        y_true.values,
        label="Actual",
        color="black",
        linewidth=3.5,
        linestyle="-"
    )

    # -----
    # Model predictions
    # -----
    for model_name, (key, color, style) in MODELS.items():
        _, y_pred = load_preds(key, metric, split)
        y_pred = y_pred.iloc[:max_points]

```

```

plt.plot(
    y_pred.values,
    label=model_name,
    color=color,
    linestyle=style,
    linewidth=2
)

plt.title(
    f"{metric} - Time Series Comparison ({split.capitalize()})",
    fontsize=13
)
plt.xlabel("Time Index")
plt.ylabel(metric)

# -----
# Legend BELOW the plot
# -----
plt.legend(
    loc="upper center",
    bbox_to_anchor=(0.5, -0.18),
    ncol=4,
    frameon=True,
    fontsize=9
)

# Extra bottom space for legend
plt.tight_layout(rect=[0, 0.08, 1, 1])

out = os.path.join(
    PLOT_DIR,
    f"compare_{metric.lower()}_{split}_time_series.png"
)
plt.savefig(out, dpi=300)
print(f"Saved → {out}")

if showImage:
    plt.show()
else:
    plt.close()

```

```

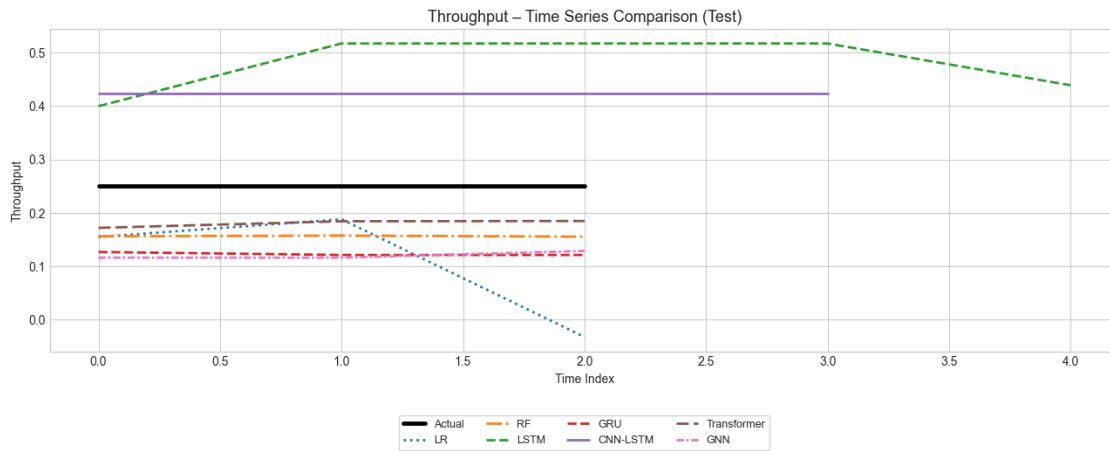
[26]: plot_model_comparison_time_series(
    metric="Throughput",
    split="test",
    max_points=300,
    showImage=True

```

)

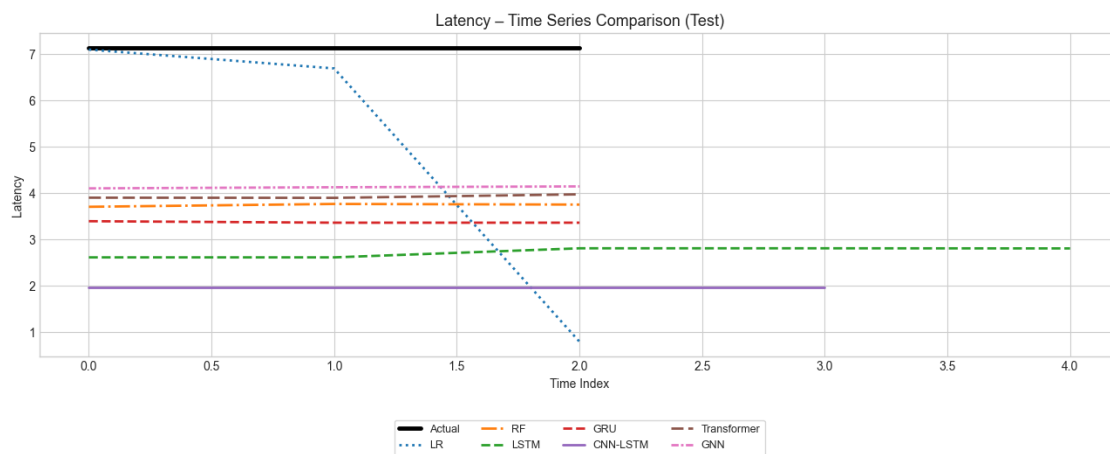
Saved →

../outputs/models/plots/comparisons/compare_throughput_test_time_series.png



```
[27]: plot_model_comparison_time_series(  
    metric="Latency",  
    split="test",  
    showImage=True  
)
```

Saved → ../outputs/models/plots/comparisons/compare_latency_test_time_series.png



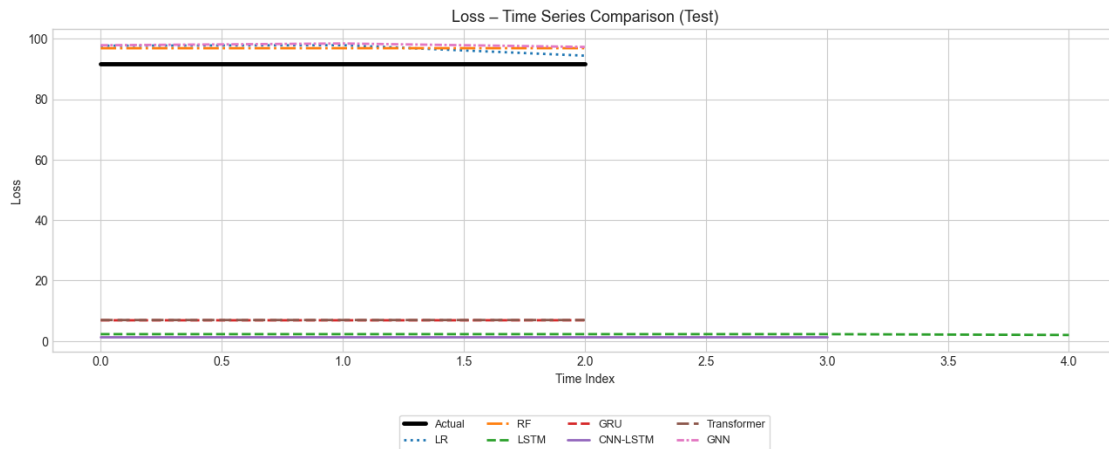
```
[28]: plot_model_comparison_time_series(  
    metric="Loss",  
    split="test",  
    showImage=True  
)
```

```

    showImage=True
)

```

Saved → ../outputs/models/plots/comparisons/compare_loss_test_time_series.png



```

[29]: import os
import pandas as pd

from edge_evaluations.lr_edge_evaluation import evaluate_lr_edge
from edge_evaluations.rf_edge_evalutaion import evaluate_rf_edge
from edge_evaluations.xgb_edge_evaluation import evaluate_xgb_edge
from edge_evaluations.mlp_edge_evaluation import evaluate_mlp_edge
from edge_evaluations.lstm_edge_evaluation import evaluate_lstm_edge
from edge_evaluations.gru_edge_evaluation import evaluate_gru_edge
from edge_evaluations.bilstm_edge_evaluation import evaluate_bilstm_edge
from edge_evaluations.transformer_edge_evaluation import evaluate_transformer_edge
from edge_evaluations.cnn_lstm_edge_evaluation import evaluate_cnn_lstm_edge
from edge_evaluations.gnn_edge_evaluation import evaluate_gnn_edge

from tqdm import tqdm
import numpy as np
import matplotlib.pyplot as plt

```

```

[30]: # import warnings
# warnings.filterwarnings("ignore", message="X does not have valid feature_
# names")

# METRIC_SUFFIX = {
#     "Throughput": "throughput",

```

```

#     "Latency": "latency",
#     "Loss": "loss",
# }

# def run_all_edge_metrics(metric="Throughput", input_dim=103):

#     assert metric in ["Throughput", "Latency", "Loss"], \
#         f"Invalid metric: {metric}"

#     metric_key = METRIC_SUFFIX[metric]

#     # -----
#     # Load feature names (for XGB)
#     # -----
#     df_train = pd.read_csv("../outputs/datasets/train.csv")
#     feature_cols = [
#         c for c in df_train.columns
#         if c not in [
#             "Perf_Avg_Throughput", "Perf_Avg_Latency", "Perf_Avg_Loss",
#             "StartDateTime", "EndDateTime", "Perf_Filename",
#             "ND_Filename", "RunDateTime"
#         ]
#     ]

#     # -----
#     # Define models + paths
#     # -----
#     edge_jobs = {

#         "LR": lambda: evaluate_lr_edge(
#             model_path=f"../outputs/models/lr_model_{metric_key}.pkl",
#             scaler_path=f"../outputs/models/lr_scaler.pkl",
#             input_dim=input_dim
#         ),

#         "RF": lambda: evaluate_rf_edge(
#             model_path=f"../outputs/models/rf_model_{metric_key}.pkl",
#             X_sample=np.random.randn(1, input_dim)
#         ),

#         "XGB": lambda: evaluate_xgb_edge(
#             model_path=f"../outputs/models/xgb_model_{metric_key}.json",
#             X_sample=pd.DataFrame(
#                 [np.random.randn(len(feature_cols))],
#                 columns=feature_cols
#             )
#     }

```

```

#         ),

#         "MLP": lambda: evaluate_mlp_edge(
#             model_path=f"../outputs/models/mlp_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "LSTM": lambda: evaluate_lstm_edge(
#             model_path=f"../outputs/models/lstm_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "GRU": lambda: evaluate_gru_edge(
#             model_path=f"../outputs/models/gru_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "BiLSTM": lambda: evaluate_bilstm_edge(
#             model_path=f"../outputs/models/bilstm_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "Transformer": lambda: evaluate_transformer_edge(
#             model_path=f"../outputs/models/transformer_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "CNN-LSTM": lambda: evaluate_cnn_lstm_edge(
#             model_path=f"../outputs/models/cnn_lstm_model_{metric_key}.pt",
#             input_dim=input_dim
#         ),

#         "GNN": lambda: evaluate_gnn_edge(
#             model_path=f"../outputs/models/gnn_model_{metric_key}.pt",
#             input_dim=3,
#             num_nodes=10
#         ),
#     }

#     # -----
#     # Run models with tqdm
#     # -----
#     print(f"\n===== Running Unified Edge Evaluation ({metric})\n",
#           "\n")

#     results = {}
#     pbar = tqdm(edge_jobs.items(), total=len(edge_jobs), unit="model")

```

```

#     for model_name, eval_fn in pbar:
#         pbar.set_description(f"Evaluating {model_name}")
#         results[model_name] = eval_fn()

#     # -----
#     # Save results
#     # -----
#     df = pd.DataFrame(results).T

#     out_dir = "../outputs/edge_analysis/"
#     os.makedirs(out_dir, exist_ok=True)

#     out_path = os.path.join(
#         out_dir,
#         f"{metric_key}_edge_metrics.csv"
#     )
#     df.to_csv(out_path)

#     print(f"\nSaved unified edge evaluation → {out_path}")
#     return df

```

```

[31]: import os
import warnings
import numpy as np
import pandas as pd
from tqdm import tqdm

warnings.filterwarnings("ignore", message="X does not have valid feature names")

# -----
# Fixed feature dimensionality (after feature selection)
# -----
FIXED_INPUT_DIM = 73

METRIC_SUFFIX = {
    "Throughput": "throughput",
    "Latency": "latency",
    "Loss": "loss",
}

def run_all_edge_metrics(metric="Throughput"):
    """
    Unified edge-efficiency evaluation for all models
    using a fixed 73-feature input space.
    """

```

```

assert metric in METRIC_SUFFIX, f"Invalid metric: {metric}"
metric_key = METRIC_SUFFIX[metric]

# -----
# Load feature names (needed for XGBoost)
# -----
df_train = pd.read_csv("../outputs/datasets/train.csv")

feature_cols = [
    c for c in df_train.columns
    if c not in [
        "Perf_Avg_Throughput",
        "Perf_Avg_Latency",
        "Perf_Avg_Loss",
        "StartDateTime",
        "EndDateTime",
        "Perf_Filename",
        "ND_Filename",
        "RunDateTime",
    ]
]

# Enforce 73 features explicitly
feature_cols = feature_cols[:FIXED_INPUT_DIM]

# -----
# Define unified edge evaluation jobs
# -----
edge_jobs = {

# -----
# Linear Regression (fixed-dim)
# -----
"LR": lambda: evaluate_lr_edge(
    model_path=f"../outputs/models/lr_model_{metric_key}.pkl",
    scaler_path=f"../outputs/models/lr_scaler.pkl",
    input_dim=FIXED_INPUT_DIM
),

# -----
# Random Forest
# -----
"RF": lambda: evaluate_rf_edge(
    model_path=f"../outputs/models/rf_model_{metric_key}.pkl",
    X_sample=np.random.randn(1, FIXED_INPUT_DIM)
),

```

```

# -----
# XGBoost
# -----
"XGB": lambda: evaluate_xgb_edge(
    model_path=f"../outputs/models/xgb_model_{metric_key}.json",
    X_sample=pd.DataFrame(
        [np.random.randn(FIXED_INPUT_DIM)],
        columns=feature_cols
    )
),

# -----
# Neural models (AUTO input_dim)
# -----
"MLP": lambda: evaluate_mlp_edge(
    model_path=f"../outputs/models/mlp_model_{metric_key}.pt"
),

"LSTM": lambda: evaluate_lstm_edge(
    model_path=f"../outputs/models/lstm_model_{metric_key}.pt"
),

"GRU": lambda: evaluate_gru_edge(
    model_path=f"../outputs/models/gru_model_{metric_key}.pt"
),

"BiLSTM": lambda: evaluate_bilstm_edge(
    model_path=f"../outputs/models/bilstm_model_{metric_key}.pt"
),

"Transformer": lambda: evaluate_transformer_edge(
    model_path=f"../outputs/models/transformer_model_{metric_key}.pt"
),

"CNN-LSTM": lambda: evaluate_cnn_lstm_edge(
    model_path=f"../outputs/models/cnn_lstm_model_{metric_key}.pt"
),

# -----
# Graph Neural Network
# -----
"GNN": lambda: evaluate_gnn_edge(
    model_path=f"../outputs/models/gnn_model_{metric_key}.pt",
    input_dim=3,
    num_nodes=10
),

```

```

}

# -----
# Run unified evaluation
# -----
print(f"\n===== Running Unified Edge Evaluation ({metric})\n")

results = {}
pbar = tqdm(edge_jobs.items(), total=len(edge_jobs), unit="model")

for model_name, eval_fn in pbar:
    pbar.set_description(f"Evaluating {model_name}")
    results[model_name] = eval_fn()

# -----
# Save results
# -----
df = pd.DataFrame(results).T

out_dir = "../outputs/edge_analysis/"
os.makedirs(out_dir, exist_ok=True)

out_path = os.path.join(
    out_dir,
    f"{metric_key}_edge_metrics.csv"
)

df.to_csv(out_path)

print(f"\nSaved unified edge evaluation → {out_path}")
return df

```

```
[32]: run_all_edge_metrics(metric="Throughput")
```

```
===== Running Unified Edge Evaluation (Throughput) =====
```

```
Evaluating GNN: 100%|          | 10/10 [00:13<00:00,  1.31s/model]
```

```
Saved unified edge evaluation →
../outputs/edge_analysis/throughput_edge_metrics.csv
```

[32]:

	Parameters	Model Size (Bytes)	Model Size (MB)	FLOPs	Latency (s)	\
LR	74	296	0.000282	146	0.000025	
RF	NaN	204161	0.194703	NaN	0.055742	
XGB	NaN	196385	0.187287	NaN	0.000385	
MLP	21633	86532	0.082523	42880	0.000177	
LSTM	76609	306436	0.29224	86144	0.000484	
GRU	51713	206852	0.197269	53120	0.000205	
BiLSTM	170625	682500	0.650883	141568	0.000513	
Transformer	406145	1624580	1.54932	393728	0.000422	
CNN-LSTM	53185	NaN	0.202885	106370	NaN	
GNN	1217	4868	0.004642	NaN	0.000694	

	Latency (ms)	Energy Estimated (J)	Peak Activation Memory (Bytes)	\
LR	0.024513	0.0	4	
RF	55.742058	0.111484	584	
XGB	0.385381	0.000771	584	
MLP	0.176796	0.000354	4	
LSTM	0.484178	0.000968	4	
GRU	0.205193	0.00041	4	
BiLSTM	0.512561	0.001025	4	
Transformer	0.422172	NaN	4	
CNN-LSTM	0.430555	0.000861	4120	
GNN	0.693521	NaN	120	

	Model Path	Trees	\
LR	../outputs/models/lr_model_throughput.pkl	NaN	
RF	../outputs/models/rf_model_throughput.pkl	300	
XGB	../outputs/models/xgb_model_throughput.json	400	
MLP	../outputs/models/mlp_model_throughput.pt	NaN	
LSTM	../outputs/models/lstm_model_throughput.pt	NaN	
GRU	../outputs/models/gru_model_throughput.pt	NaN	
BiLSTM	../outputs/models/bilstm_model_throughput.pt	NaN	
Transformer	../outputs/models/transformer_model_throughput.pt	NaN	
CNN-LSTM	../outputs/models/cnn_lstm_model_throughput.pt	NaN	
GNN	../outputs/models/gnn_model_throughput.pt	NaN	

	Total Nodes	Average Nodes per Tree	Average Tree Depth	\
LR	NaN	NaN	NaN	
RF	1500	5.0	2.0	
XGB	912	2.28	0.625	
MLP	NaN	NaN	NaN	
LSTM	NaN	NaN	NaN	
GRU	NaN	NaN	NaN	
BiLSTM	NaN	NaN	NaN	
Transformer	NaN	NaN	NaN	
CNN-LSTM	NaN	NaN	NaN	
GNN	NaN	NaN	NaN	

	Max Tree Depth	Input Dim	Estimated Energy (J)	Num Nodes	\
LR	NaN	NaN	NaN	NaN	
RF	2	NaN	NaN	NaN	
XGB	4	NaN	NaN	NaN	
MLP	NaN	103	NaN	NaN	
LSTM	NaN	103	NaN	NaN	
GRU	NaN	73	NaN	NaN	
BiLSTM	NaN	73	NaN	NaN	
Transformer	NaN	73	0.000844	NaN	
CNN-LSTM	NaN	103	NaN	NaN	
GNN	NaN	NaN	0.003468	10	

	Node Feature Dim	Hidden Dim
LR	NaN	NaN
RF	NaN	NaN
XGB	NaN	NaN
MLP	NaN	NaN
LSTM	NaN	NaN
GRU	NaN	NaN
BiLSTM	NaN	NaN
Transformer	NaN	NaN
CNN-LSTM	NaN	NaN
GNN	3	32

```
[33]: run_all_edge_metrics(metric="Latency")
```

```
===== Running Unified Edge Evaluation (Latency) =====
```

```
Evaluating GNN: 100%|      | 10/10 [00:12<00:00,  1.29s/model]
```

```
Saved unified edge evaluation →  
../outputs/edge_analysis/latency_edge_metrics.csv
```

```
[33]:
```

	Parameters	Model Size (Bytes)	Model Size (MB)	FLOPs	Latency (s)	\
LR	74	296	0.000282	146	0.000016	
RF	NaN	223313	0.212968	NaN	0.051414	
XGB	NaN	246483	0.235065	NaN	0.000182	
MLP	21633	86532	0.082523	42880	0.000066	
LSTM	76609	306436	0.29224	86144	0.000311	
GRU	51713	206852	0.197269	53120	0.000186	
BiLSTM	170625	682500	0.650883	141568	0.000533	
Transformer	406145	1624580	1.54932	393728	0.000442	
CNN-LSTM	53185	NaN	0.202885	106370	NaN	

GNN	1217	4868	0.004642	NaN	0.000696
-----	------	------	----------	-----	----------

	Latency (ms)	Energy Estimated (J)	Peak Activation	Memory (Bytes)	\
LR	0.016471	0.0			4
RF	51.414001	0.102828			584
XGB	0.182134	0.000364			584
MLP	0.065858	0.000132			4
LSTM	0.310661	0.000621			4
GRU	0.186465	0.000373			4
BiLSTM	0.532998	0.001066			4
Transformer	0.441553	NaN			4
CNN-LSTM	0.423471	0.000847			4120
GNN	0.696391	NaN			120

	Model Path	Trees	Total Nodes	\
LR	../outputs/models/lr_model_latency.pkl	NaN	NaN	
RF	../outputs/models/rf_model_latency.pkl	300	1766	
XGB	../outputs/models/xgb_model_latency.json	400	1916	
MLP	../outputs/models/mlp_model_latency.pt	NaN	NaN	
LSTM	../outputs/models/lstm_model_latency.pt	NaN	NaN	
GRU	../outputs/models/gru_model_latency.pt	NaN	NaN	
BiLSTM	../outputs/models/bilstm_model_latency.pt	NaN	NaN	
Transformer	../outputs/models/transformer_model_latency.pt	NaN	NaN	
CNN-LSTM	../outputs/models/cnn_lstm_model_latency.pt	NaN	NaN	
GNN	../outputs/models/gnn_model_latency.pt	NaN	NaN	

	Average Nodes per Tree	Average Tree Depth	Max Tree Depth	\
LR	NaN	NaN	NaN	
RF	5.886667	2.373333	3	
XGB	4.79	1.7575	4	
MLP	NaN	NaN	NaN	
LSTM	NaN	NaN	NaN	
GRU	NaN	NaN	NaN	
BiLSTM	NaN	NaN	NaN	
Transformer	NaN	NaN	NaN	
CNN-LSTM	NaN	NaN	NaN	
GNN	NaN	NaN	NaN	

	Input Dim	Estimated Energy (J)	Num Nodes	Node Feature Dim	\
LR	NaN	NaN	NaN	NaN	
RF	NaN	NaN	NaN	NaN	
XGB	NaN	NaN	NaN	NaN	
MLP	103	NaN	NaN	NaN	
LSTM	103	NaN	NaN	NaN	
GRU	73	NaN	NaN	NaN	
BiLSTM	73	NaN	NaN	NaN	
Transformer	73	0.000883	NaN	NaN	

CNN-LSTM	103	NaN	NaN	NaN
GNN	NaN	0.003482	10	3

	Hidden Dim
LR	NaN
RF	NaN
XGB	NaN
MLP	NaN
LSTM	NaN
GRU	NaN
BiLSTM	NaN
Transformer	NaN
CNN-LSTM	NaN
GNN	32

```
[34]: run_all_edge_metrics(metric="Loss")
```

===== Running Unified Edge Evaluation (Loss) =====

Evaluating GNN: 100%| | 10/10 [00:13<00:00, 1.33s/model]

Saved unified edge evaluation → ../outputs/edge_analysis/loss_edge_metrics.csv

```
[34]:
```

	Parameters	Model Size (Bytes)	Model Size (MB)	FLOPs	Latency (s)	\
LR	74	296	0.000282	146	0.000009	
RF	NaN	221297	0.211045	NaN	0.056424	
XGB	NaN	231818	0.221079	NaN	0.000169	
MLP	21633	86532	0.082523	42880	0.000059	
LSTM	76609	306436	0.29224	86144	0.000287	
GRU	51713	206852	0.197269	53120	0.000198	
BiLSTM	170625	682500	0.650883	141568	0.000538	
Transformer	406145	1624580	1.54932	393728	0.000441	
CNN-LSTM	53185	NaN	0.202885	106370	NaN	
GNN	1217	4868	0.004642	NaN	0.000718	

	Latency (ms)	Energy Estimated (J)	Peak Activation Memory (Bytes)	\
LR	0.008514	0.0	4	
RF	56.423753	0.112848	584	
XGB	0.168729	0.000337	584	
MLP	0.058893	0.000118	4	
LSTM	0.287151	0.000574	4	
GRU	0.19786	0.000396	4	
BiLSTM	0.538426	0.001077	4	
Transformer	0.441023	NaN	4	

CNN-LSTM	0.414976	0.00083	4120
GNN	0.718335	NaN	120

	Model Path	Trees	Total Nodes \
LR	../outputs/models/lr_model_loss.pkl	NaN	NaN
RF	../outputs/models/rf_model_loss.pkl	300	1738
XGB	../outputs/models/xgb_model_loss.json	400	1628
MLP	../outputs/models/mlp_model_loss.pt	NaN	NaN
LSTM	../outputs/models/lstm_model_loss.pt	NaN	NaN
GRU	../outputs/models/gru_model_loss.pt	NaN	NaN
BiLSTM	../outputs/models/bilstm_model_loss.pt	NaN	NaN
Transformer	../outputs/models/transformer_model_loss.pt	NaN	NaN
CNN-LSTM	../outputs/models/cnn_lstm_model_loss.pt	NaN	NaN
GNN	../outputs/models/gnn_model_loss.pt	NaN	NaN

	Average Nodes per Tree	Average Tree Depth	Max Tree Depth \
LR	NaN	NaN	NaN
RF	5.793333	2.346667	4
XGB	4.07	1.4375	4
MLP	NaN	NaN	NaN
LSTM	NaN	NaN	NaN
GRU	NaN	NaN	NaN
BiLSTM	NaN	NaN	NaN
Transformer	NaN	NaN	NaN
CNN-LSTM	NaN	NaN	NaN
GNN	NaN	NaN	NaN

	Input Dim	Estimated Energy (J)	Num Nodes	Node Feature Dim \
LR	NaN	NaN	NaN	NaN
RF	NaN	NaN	NaN	NaN
XGB	NaN	NaN	NaN	NaN
MLP	103	NaN	NaN	NaN
LSTM	103	NaN	NaN	NaN
GRU	73	NaN	NaN	NaN
BiLSTM	73	NaN	NaN	NaN
Transformer	73	0.000882	NaN	NaN
CNN-LSTM	103	NaN	NaN	NaN
GNN	NaN	0.003592	10	3

	Hidden Dim
LR	NaN
RF	NaN
XGB	NaN
MLP	NaN
LSTM	NaN
GRU	NaN
BiLSTM	NaN

Transformer	NaN
CNN-LSTM	NaN
GNN	32

```
[35]: df=pd.read_csv("../outputs/edge_analysis/throughput_edge_metrics.csv",
    ↪index_col=0)
df.head(10)
```

```
[35]:
```

	Parameters	Model Size (Bytes)	Model Size (MB)	FLOPs \
LR	74.0	296.0	0.000282	146.0
RF	NaN	204161.0	0.194703	NaN
XGB	NaN	196385.0	0.187287	NaN
MLP	21633.0	86532.0	0.082523	42880.0
LSTM	76609.0	306436.0	0.292240	86144.0
GRU	51713.0	206852.0	0.197269	53120.0
BiLSTM	170625.0	682500.0	0.650883	141568.0
Transformer	406145.0	1624580.0	1.549320	393728.0
CNN-LSTM	53185.0	NaN	0.202885	106370.0
GNN	1217.0	4868.0	0.004642	NaN

	Latency (s)	Latency (ms)	Energy Estimated (J) \
LR	0.000025	0.024513	7.300000e-08
RF	0.055742	55.742058	1.114841e-01
XGB	0.000385	0.385381	7.707628e-04
MLP	0.000177	0.176796	3.535921e-04
LSTM	0.000484	0.484178	9.683553e-04
GRU	0.000205	0.205193	4.103861e-04
BiLSTM	0.000513	0.512561	1.025123e-03
Transformer	0.000422	0.422172	NaN
CNN-LSTM	NaN	0.430555	8.611104e-04
GNN	0.000694	0.693521	NaN

	Peak Activation Memory (Bytes) \
LR	4
RF	584
XGB	584
MLP	4
LSTM	4
GRU	4
BiLSTM	4
Transformer	4
CNN-LSTM	4120
GNN	120

	Model Path	Trees \
LR	../outputs/models/lr_model_throughput.pkl	NaN
RF	../outputs/models/rf_model_throughput.pkl	300.0

XGB	../outputs/models/xgb_model_throughput.json	400.0
MLP	../outputs/models/mlp_model_throughput.pt	NaN
LSTM	../outputs/models/lstm_model_throughput.pt	NaN
GRU	../outputs/models/gru_model_throughput.pt	NaN
BiLSTM	../outputs/models/bilstm_model_throughput.pt	NaN
Transformer	../outputs/models/transformer_model_throughput.pt	NaN
CNN-LSTM	../outputs/models/cnn_lstm_model_throughput.pt	NaN
GNN	../outputs/models/gnn_model_throughput.pt	NaN

	Total Nodes	Average Nodes per Tree	Average Tree Depth	\
LR	NaN	NaN	NaN	
RF	1500.0	5.00	2.000	
XGB	912.0	2.28	0.625	
MLP	NaN	NaN	NaN	
LSTM	NaN	NaN	NaN	
GRU	NaN	NaN	NaN	
BiLSTM	NaN	NaN	NaN	
Transformer	NaN	NaN	NaN	
CNN-LSTM	NaN	NaN	NaN	
GNN	NaN	NaN	NaN	

	Max Tree Depth	Input Dim	Estimated Energy (J)	Num Nodes	\
LR	NaN	NaN	NaN	NaN	
RF	2.0	NaN	NaN	NaN	
XGB	4.0	NaN	NaN	NaN	
MLP	NaN	103.0	NaN	NaN	
LSTM	NaN	103.0	NaN	NaN	
GRU	NaN	73.0	NaN	NaN	
BiLSTM	NaN	73.0	NaN	NaN	
Transformer	NaN	73.0	0.000844	NaN	
CNN-LSTM	NaN	103.0	NaN	NaN	
GNN	NaN	NaN	0.003468	10.0	

	Node Feature Dim	Hidden Dim
LR	NaN	NaN
RF	NaN	NaN
XGB	NaN	NaN
MLP	NaN	NaN
LSTM	NaN	NaN
GRU	NaN	NaN
BiLSTM	NaN	NaN
Transformer	NaN	NaN
CNN-LSTM	NaN	NaN
GNN	3.0	32.0

```
[36]: df.describe()
```

[36]:	Parameters	Model Size (Bytes)	Model Size (MB)	FLOPs	\
count	8.000000	9.000000e+00	10.000000	7.000000	
mean	97650.125000	3.680678e+05	0.336204	117708.000000	
std	136079.581274	5.135967e+05	0.464161	130046.683613	
min	74.000000	2.960000e+02	0.000282	146.000000	
25%	16529.000000	8.653200e+04	0.108714	48000.000000	
50%	52449.000000	2.041610e+05	0.195986	86144.000000	
75%	100113.000000	3.064360e+05	0.269901	123969.000000	
max	406145.000000	1.624580e+06	1.549320	393728.000000	
	Latency (s)	Latency (ms)	Energy Estimated (J)	\	
count	9.000000	10.000000	8.000000e+00		
mean	0.006516	5.907693	1.448419e-02		
std	0.018461	17.511053	3.919547e-02		
min	0.000025	0.024513	7.300000e-08		
25%	0.000205	0.250240	3.961876e-04		
50%	0.000422	0.426363	8.159366e-04		
75%	0.000513	0.505465	9.825471e-04		
max	0.055742	55.742058	1.114841e-01		
	Peak Activation Memory (Bytes)	Trees	Total Nodes	\	
count	10.000000	2.000000	2.000000		
mean	543.200000	350.000000	1206.000000		
std	1278.903419	70.710678	415.778787		
min	4.000000	300.000000	912.000000		
25%	4.000000	325.000000	1059.000000		
50%	4.000000	350.000000	1206.000000		
75%	468.000000	375.000000	1353.000000		
max	4120.000000	400.000000	1500.000000		
	Average Nodes per Tree	Average Tree Depth	Max Tree Depth	Input Dim	\
count	2.00000	2.000000	2.000000	6.000000	
mean	3.64000	1.312500	3.000000	88.000000	
std	1.92333	0.972272	1.414214	16.431677	
min	2.28000	0.625000	2.000000	73.000000	
25%	2.96000	0.968750	2.500000	73.000000	
50%	3.64000	1.312500	3.000000	88.000000	
75%	4.32000	1.656250	3.500000	103.000000	
max	5.00000	2.000000	4.000000	103.000000	
	Estimated Energy (J)	Num Nodes	Node Feature Dim	Hidden Dim	
count	2.000000	1.0	1.0	1.0	
mean	0.002156	10.0	3.0	32.0	
std	0.001855	NaN	NaN	NaN	
min	0.000844	10.0	3.0	32.0	
25%	0.001500	10.0	3.0	32.0	
50%	0.002156	10.0	3.0	32.0	

75%	0.002812	10.0	3.0	32.0
max	0.003468	10.0	3.0	32.0

```
[37]: print(df.columns.tolist())
```

```
['Parameters', 'Model Size (Bytes)', 'Model Size (MB)', 'FLOPs', 'Latency (s)',
'Latency (ms)', 'Energy Estimated (J)', 'Peak Activation Memory (Bytes)', 'Model
Path', 'Trees', 'Total Nodes', 'Average Nodes per Tree', 'Average Tree Depth',
'Max Tree Depth', 'Input Dim', 'Estimated Energy (J)', 'Num Nodes', 'Node
Feature Dim', 'Hidden Dim']
```

```
[38]: import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

METRIC_SUFFIX = {
    "Throughput": "throughput",
    "Latency": "latency",
    "Loss": "loss",
}

def load_metrics(metric="Throughput"):
    assert metric in METRIC_SUFFIX, f"Invalid metric: {metric}"

    metric_key = METRIC_SUFFIX[metric]
    csv_path = f"../outputs/edge_analysis/{metric_key}_edge_metrics.csv"

    df = pd.read_csv(csv_path, index_col=0)

    # Validate required columns
    required = [
        "Model Size (MB)",
        "Latency (ms)",
        "Energy Estimated (J)",
        "FLOPs",
    ]

    missing = [c for c in required if c not in df.columns]
    if missing:
        raise KeyError(
            f"Missing required columns: {missing}\n"
            f"Available: {df.columns.tolist()}"
        )

    return df
```

```

def plot_modelsize(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    plt.bar(df.index, df["Model Size (MB)"], alpha=0.6, label="Model Size (MB)")
    # plt.plot(df.index, df["Latency (ms)"], color="red", marker="o",
    ↪ linewidth=2, label="Latency (ms)")

    plt.title(f"{metric} - Model Size Comparison")
    plt.xlabel("Model")
    plt.ylabel("Value")
    plt.grid(True, linestyle="--", alpha=0.4)
    plt.legend()

    path = os.path.join(save_dir, "modelsize.png")
    plt.savefig(path, dpi=300, bbox_inches="tight")
    if showImage:
        plt.show()
    plt.close()

def plot_size_vs_latency(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    plt.bar(df.index, df["Model Size (MB)"], alpha=0.6, label="Model Size (MB)")
    plt.plot(df.index, df["Latency (ms)"], color="red", marker="o",
    ↪ linewidth=2, label="Latency (ms)")

    plt.title(f"{metric} - Model Size vs Inference Latency")
    plt.xlabel("Model")
    plt.ylabel("Value")
    plt.grid(True, linestyle="--", alpha=0.4)
    plt.legend()

    path = os.path.join(save_dir, "size_vs_latency.png")
    plt.savefig(path, dpi=300, bbox_inches="tight")
    if showImage:
        plt.show()
    plt.close()

def plot_latency(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    # plt.bar(df.index, df["Model Size (MB)"], alpha=0.6, label="Model Size
    ↪ (MB)")

```

```

plt.plot(df.index, df["Latency (ms)"], color="red", marker="o",
↪linewidth=2, label="Latency (ms)")

plt.title(f"{metric} - Inference Latency")
plt.xlabel("Model")
plt.ylabel("Value")
plt.grid(True, linestyle="--", alpha=0.4)
plt.legend()

path = os.path.join(save_dir, "inference_latency.png")
plt.savefig(path, dpi=300, bbox_inches="tight")
if showImage:
    plt.show()
plt.close()

def plot_energy(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    plt.bar(df.index, df["Energy Estimated (J)"], color="orange", alpha=0.7)

    plt.title("Energy per Inference (J)")
    plt.xlabel("Model")
    plt.ylabel("Energy (J)")
    plt.grid(True, linestyle="--", alpha=0.4)

    path = os.path.join(save_dir, "energy_per_inference.png")
    plt.savefig(path, dpi=300, bbox_inches="tight")
    if showImage:
        plt.show()
    plt.close()

def plot_flops(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    plt.bar(df.index, df["FLOPs"], color="green", alpha=0.7)

    plt.title(f"{metric} - Model FLOPs per Inference")
    plt.xlabel("Model")
    plt.ylabel("FLOPs")
    plt.grid(True, linestyle="--", alpha=0.4)

    path = os.path.join(save_dir, "flops_comparison.png")
    plt.savefig(path, dpi=300, bbox_inches="tight")
    if showImage:
        plt.show()

```

```

plt.close()

def plot_radar(df, save_dir, metric="Throughput", showImage=False):

    metrics = ["Model Size (MB)", "Latency (ms)", "Energy Estimated (J)",
    ↪ "FLOPs"]
    labels = df.index.tolist()

    # normalization 0-1
    norm_df = (df[metrics] - df[metrics].min()) / (df[metrics].max() -
    ↪ df[metrics].min())

    angles = np.linspace(0, 2*np.pi, len(metrics), endpoint=False).tolist()
    angles += angles[:1]

    plt.figure(figsize=(8, 8))
    ax = plt.subplot(111, polar=True)

    for model in labels:
        values = norm_df.loc[model].tolist()
        values += values[:1]

        ax.plot(angles, values, linewidth=2, label=model)
        ax.fill(angles, values, alpha=0.1)

    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(metrics)
    plt.title(f"{metric} - Normalized Model Comparison (Radar Chart)")
    plt.legend(bbox_to_anchor=(1.2, 1.1))

    path = os.path.join(save_dir, "radar_comparison.png")
    plt.savefig(path, dpi=300, bbox_inches="tight")
    if showImage:
        plt.show()
    plt.close()

def plot_latency_energy(df, save_dir, metric="Throughput", showImage=False):
    plt.figure(figsize=(10, 6))

    x = df["Latency (ms)"]
    y = df["Energy Estimated (J)"]
    sizes = df["Model Size (MB)"] * 25

    plt.scatter(x, y, s=sizes, alpha=0.75)

```

```

for model in df.index:
    plt.annotate(
        model,
        (x[model], y[model]),
        textcoords="offset points",
        xytext=(5, 5),
        fontsize=9
    )

# -----
# Log scaling (key improvement)
# -----
plt.xscale("log")
plt.yscale("log")

plt.title(f"{metric} - Latency vs Energy per Inference (Bubble = Model_
↳Size)")
plt.xlabel("Latency (ms, log scale)")
plt.ylabel("Energy per Inference (J, log scale)")
plt.grid(True, which="both", linestyle="--", alpha=0.4)

path = os.path.join(save_dir, "latency_vs_energy_log.png")
plt.savefig(path, dpi=300, bbox_inches="tight")

if showImage:
    plt.show()
plt.close()

def plot_latency_energy_barline(
    df,
    save_dir,
    metric_name="Throughput",
    showImage=False
):
    import numpy as np

    models = df.index.tolist()
    x = np.arange(len(models))

    latency = df["Latency (ms)"]
    energy = df["Energy Estimated (J)"]

    # -----
    # Identify best model (edge-efficient)
    # -----
    best_model = (

```

```

        df.sort_values(
            by=["Latency (ms)", "Energy Estimated (J)"],
            ascending=[True, True]
        )
        .index[0]
    )
    best_idx = models.index(best_model)

    # -----
    # Plot
    # -----
    fig, ax1 = plt.subplots(figsize=(11, 6))

    # Latency bars
    bars = ax1.bar(
        x,
        latency,
        width=0.6,
        color="#4C72B0",
        alpha=0.8,
        label="Latency (ms)"
    )

    # Highlight best latency bar
    bars[best_idx].set_edgecolor("red")
    bars[best_idx].set_linewidth(2.5)

    ax1.set_xlabel("Model")
    ax1.set_ylabel("Latency (ms)", color="#4C72B0")
    ax1.tick_params(axis="y", labelcolor="#4C72B0")
    ax1.set_xticks(x)
    ax1.set_xticklabels(models, rotation=30, ha="right")
    ax1.grid(True, axis="y", linestyle="--", alpha=0.4)

    # -----
    # Energy line (secondary axis)
    # -----
    ax2 = ax1.twinx()
    ax2.plot(
        x,
        energy,
        color="#DD8452",
        marker="o",
        linewidth=2.5,
        label="Energy per Inference (J)"
    )

```

```

# Highlight best energy point
ax2.scatter(
    x[best_idx],
    energy.iloc[best_idx],
    s=120,
    facecolors="none",
    edgecolors="red",
    linewidths=2.5,
    zorder=10
)

ax2.set_ylabel("Energy per Inference (J)", color="#DD8452")
ax2.tick_params(axis="y", labelcolor="#DD8452")

# -----
# Title + annotation
# -----
plt.title(
    f"{metric_name} - Latency and Energy per Inference\n"
    f"Best Edge Model: {best_model}",
    pad=12
)

# -----
# Unified legend BELOW plot
# -----
lines, labels = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()

fig.legend(
    lines + lines2,
    labels + labels2,
    loc="upper center",
    bbox_to_anchor=(0.5, -0.18),
    ncol=2,
    frameon=False
)

# -----
# Save
# -----
os.makedirs(save_dir, exist_ok=True)
path = os.path.join(
    save_dir,
    f"{metric_name.lower()}_latency_energy_barline.png"
)

```

```

plt.tight_layout()
plt.savefig(path, dpi=300, bbox_inches="tight")

if showImage:
    plt.show()
plt.close()

def generate_edge_comparison_plots(metric="Throughput", showImage=False):

    assert metric in METRIC_SUFFIX, f"Invalid metric: {metric}"

    metric_key = METRIC_SUFFIX[metric]

    save_dir = f"../outputs/edge_analysis/plots/{metric_key}/"
    os.makedirs(save_dir, exist_ok=True)

    df = load_metrics(metric)

    plot_modelsize(df, save_dir, metric, showImage)
    plot_size_vs_latency(df, save_dir, metric, showImage)
    plot_latency(df, save_dir, metric, showImage)
    plot_energy(df, save_dir, metric, showImage)
    plot_flops(df, save_dir, metric, showImage)
    plot_radar(df, save_dir, metric, showImage)
    plot_latency_energy(df, save_dir, metric, showImage)
    plot_latency_energy_barline(df, save_dir, metric, showImage)

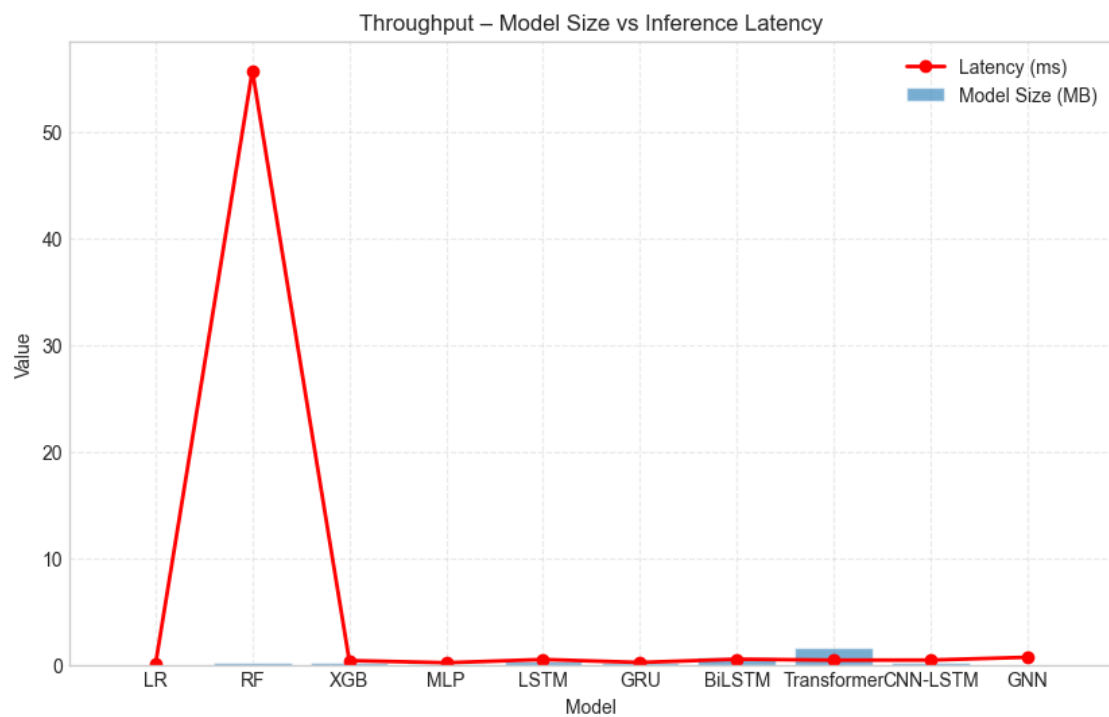
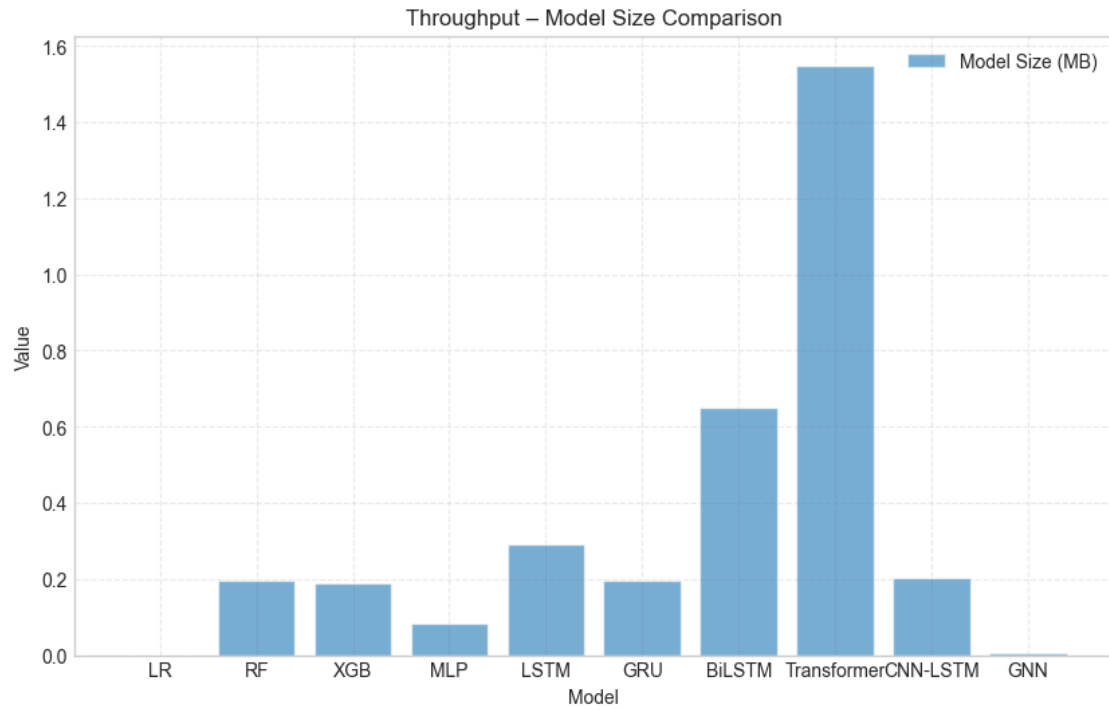
    print(f"\n All {metric} edge comparison plots generated successfully.")

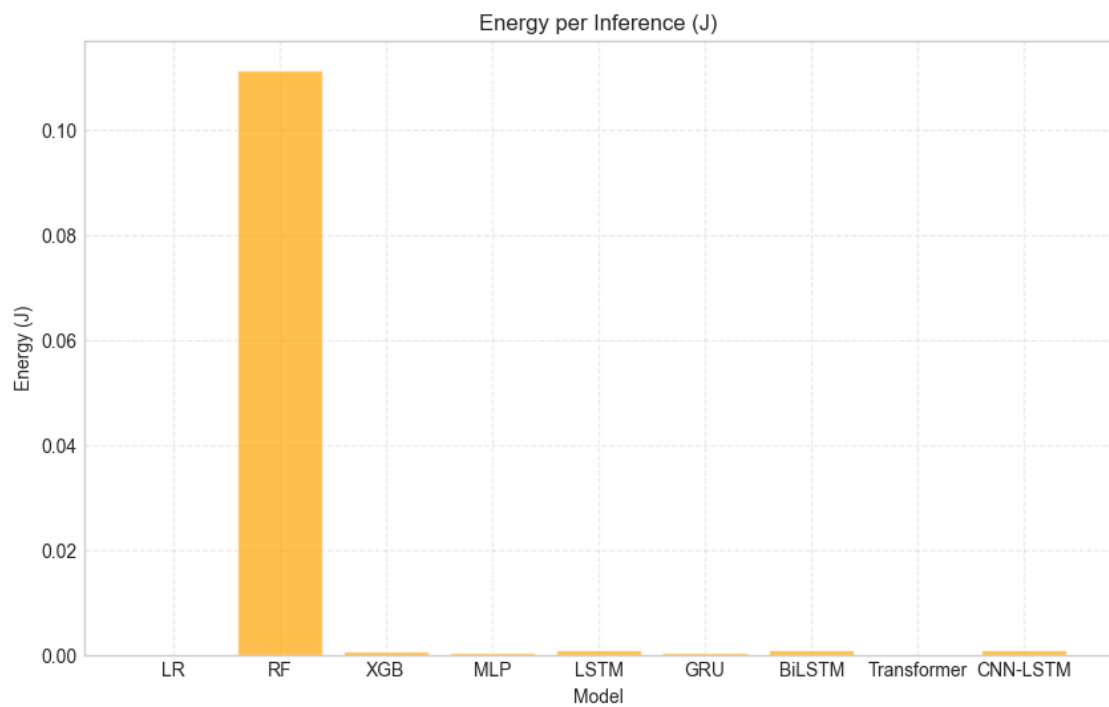
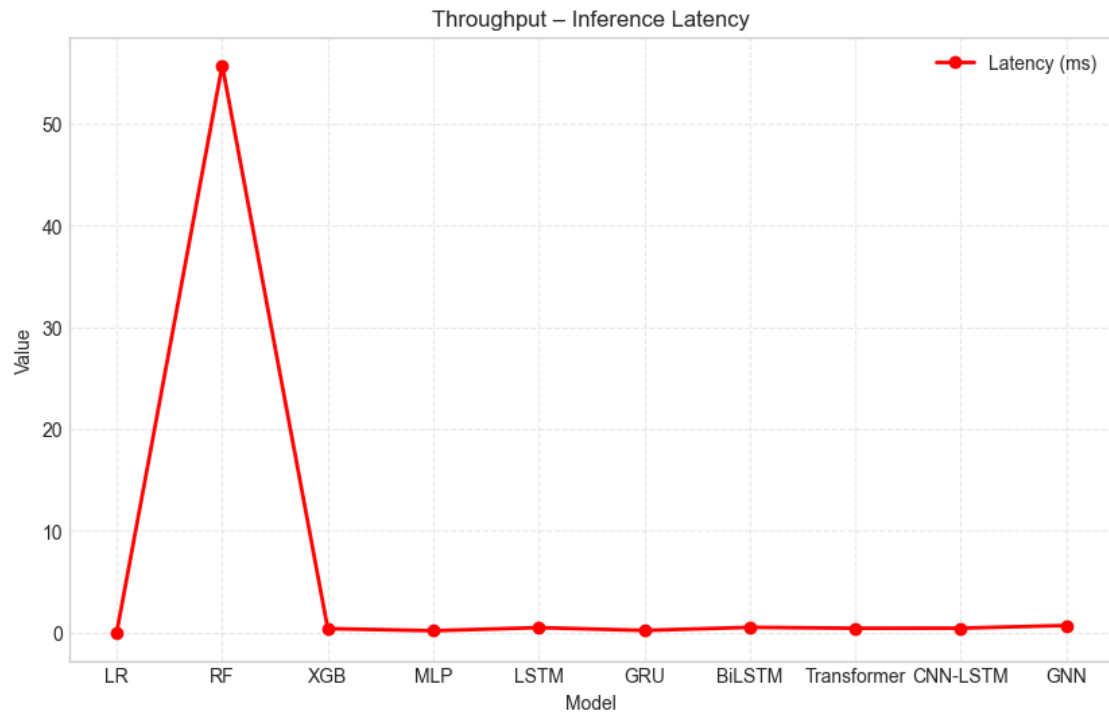
```

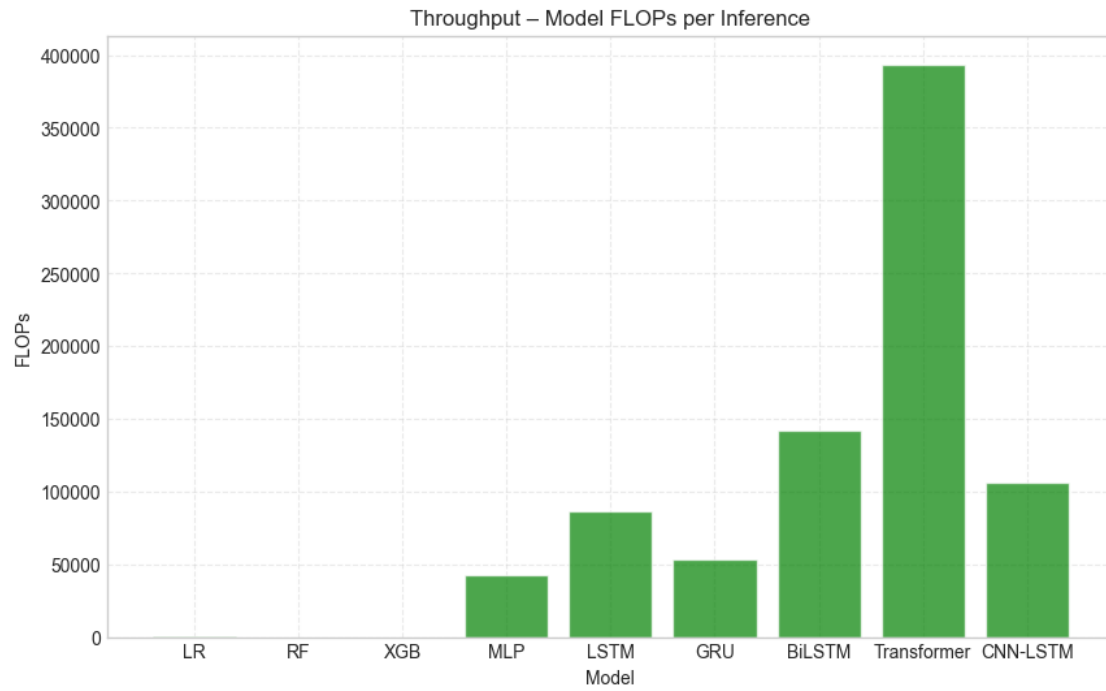
```

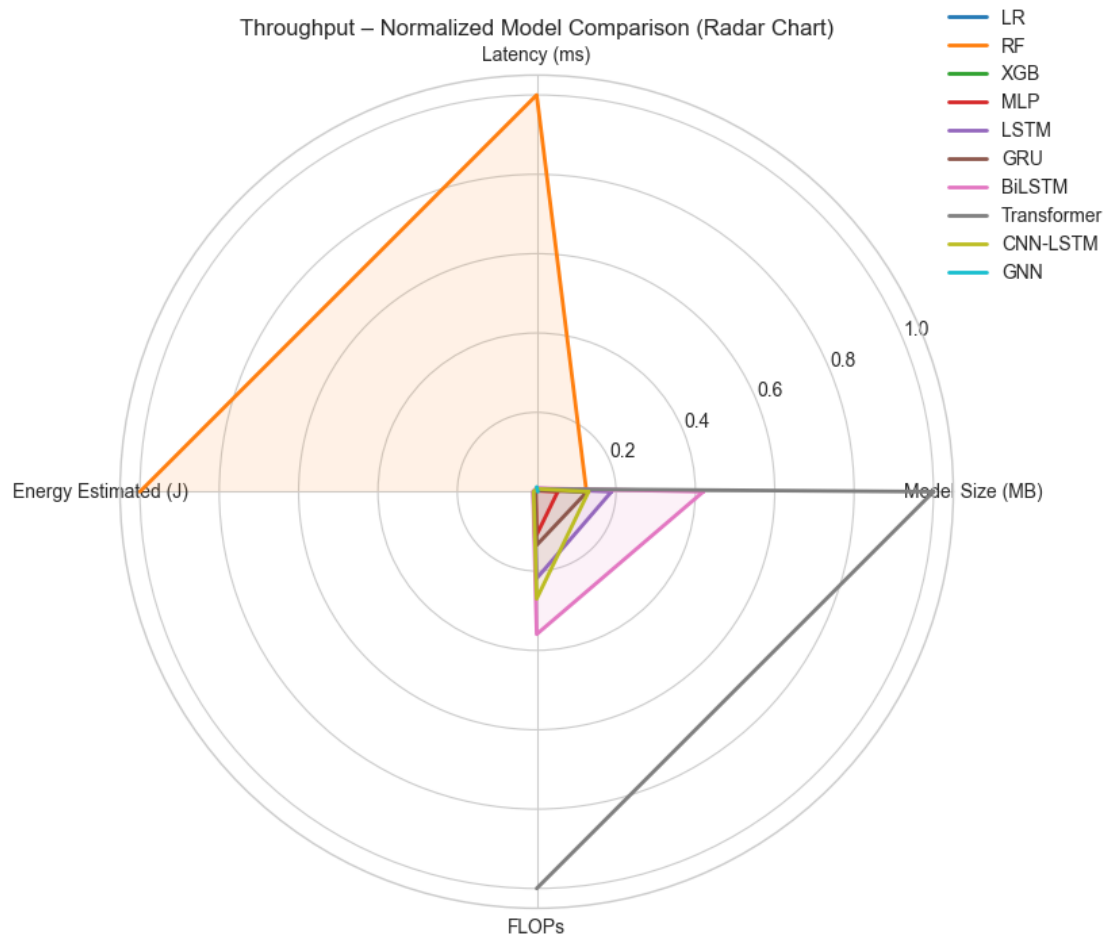
[39]: # Throughput plots
generate_edge_comparison_plots(metric="Throughput", showImage=True)

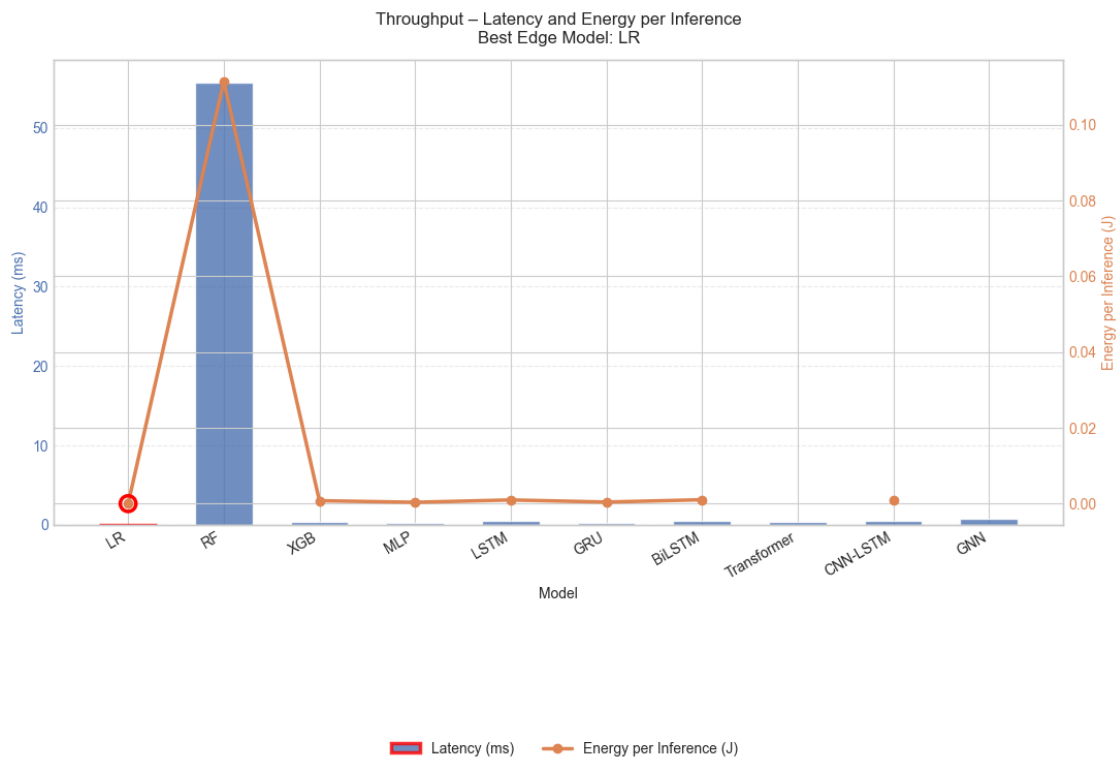
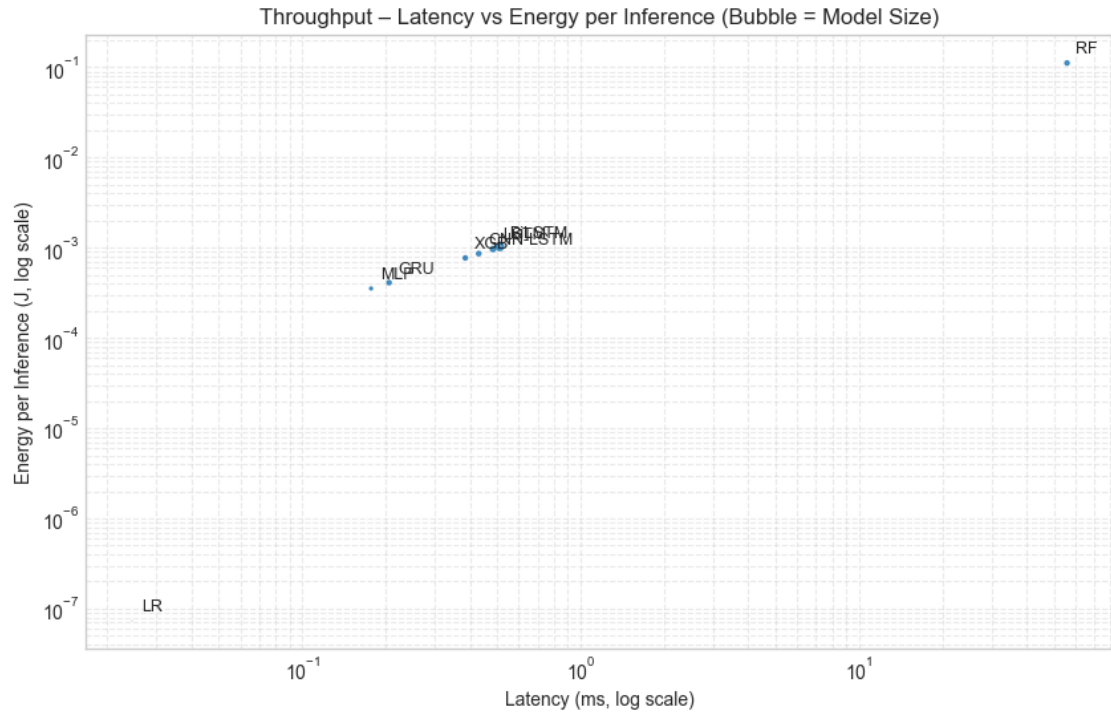
```





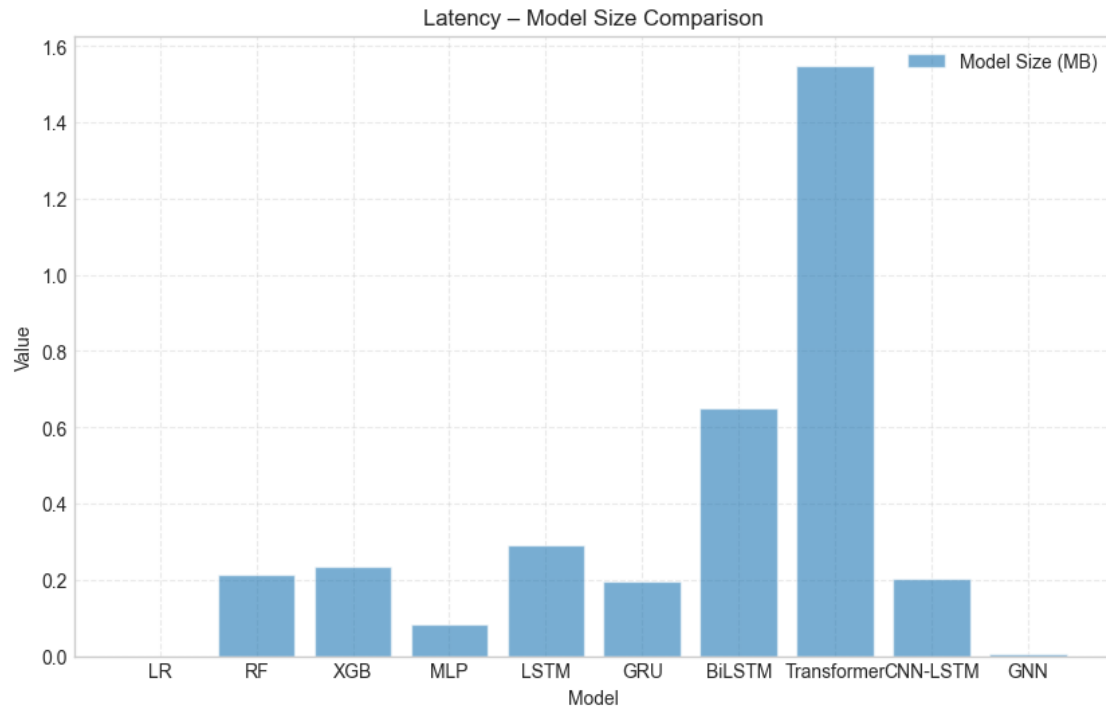


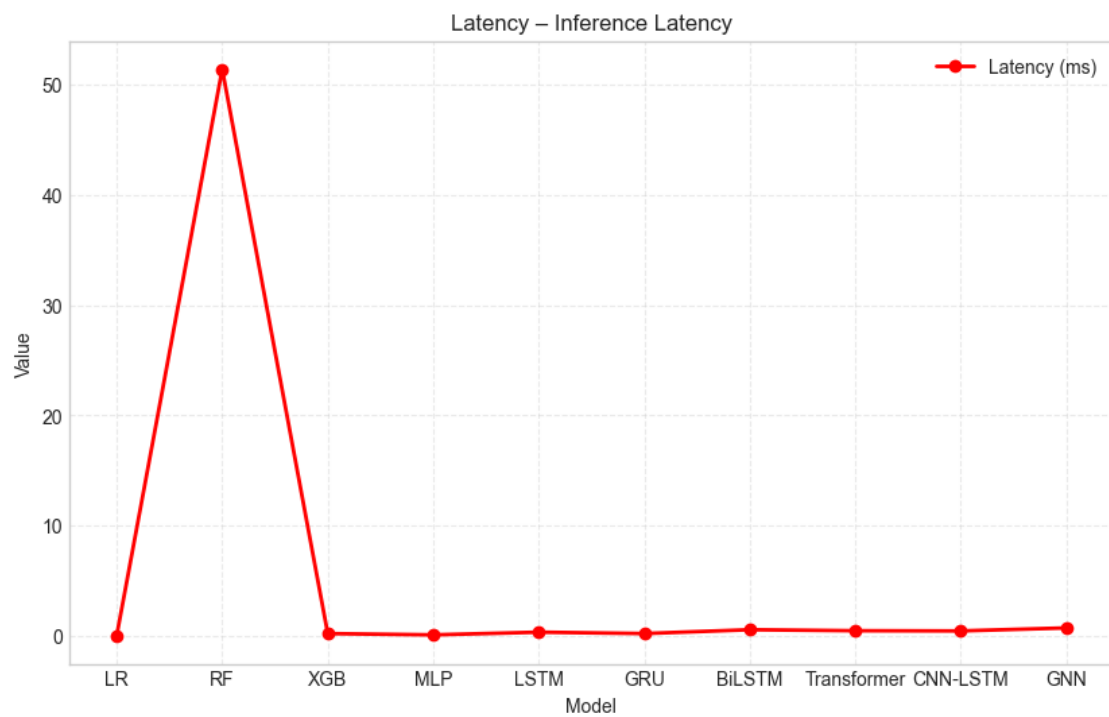
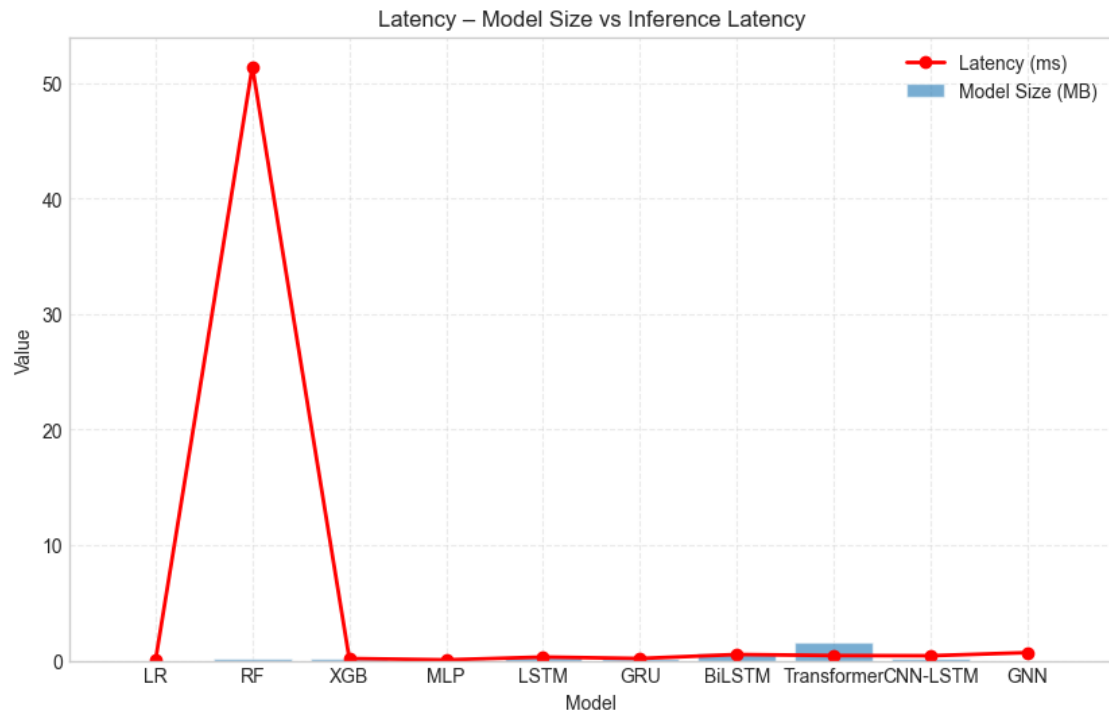


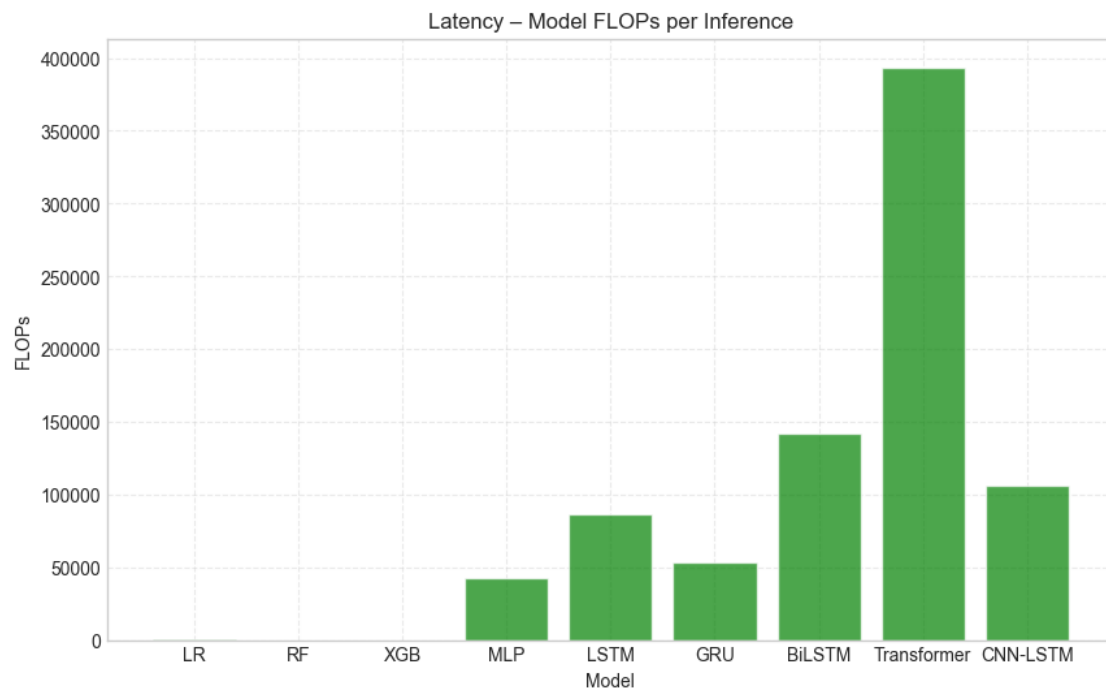
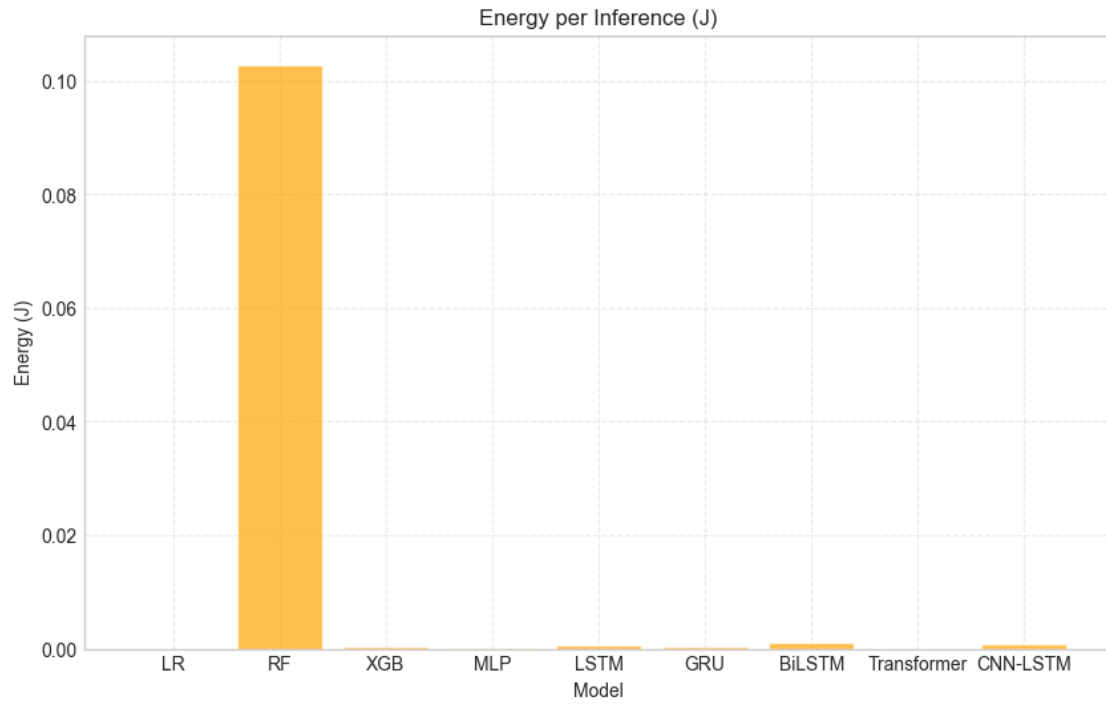


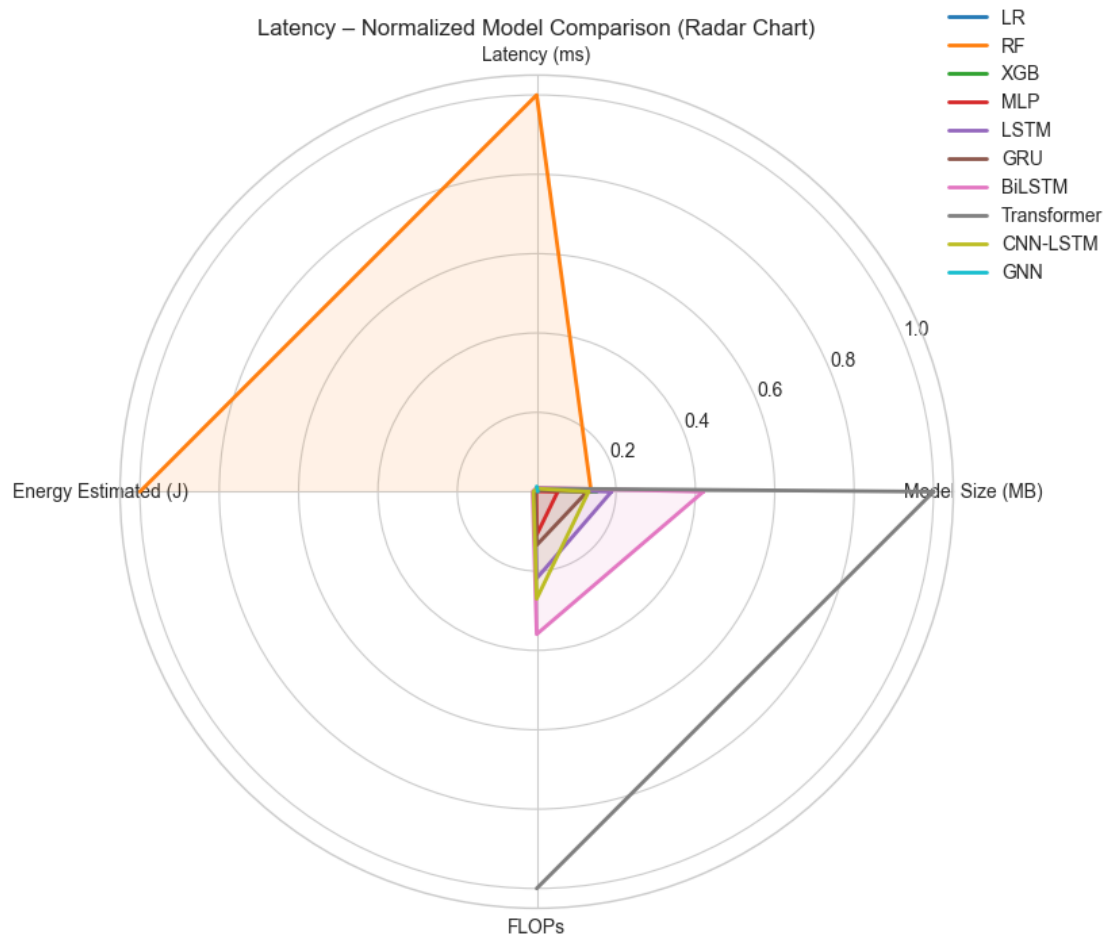
All Throughput edge comparison plots generated successfully.

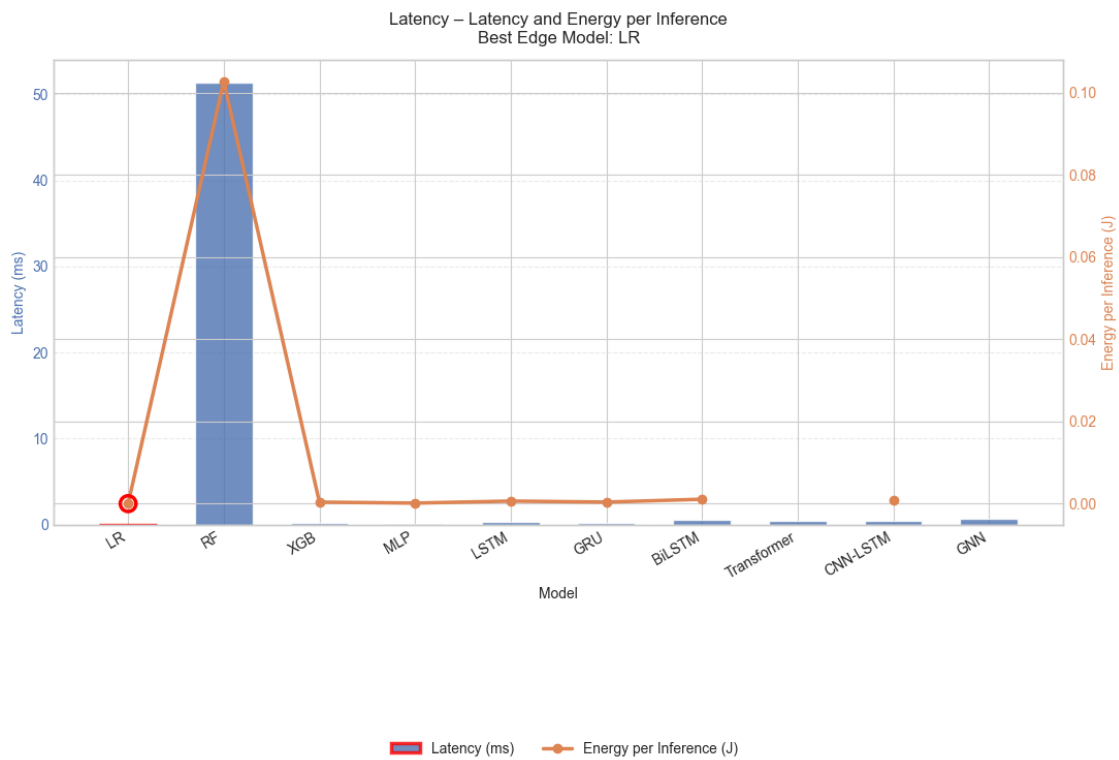
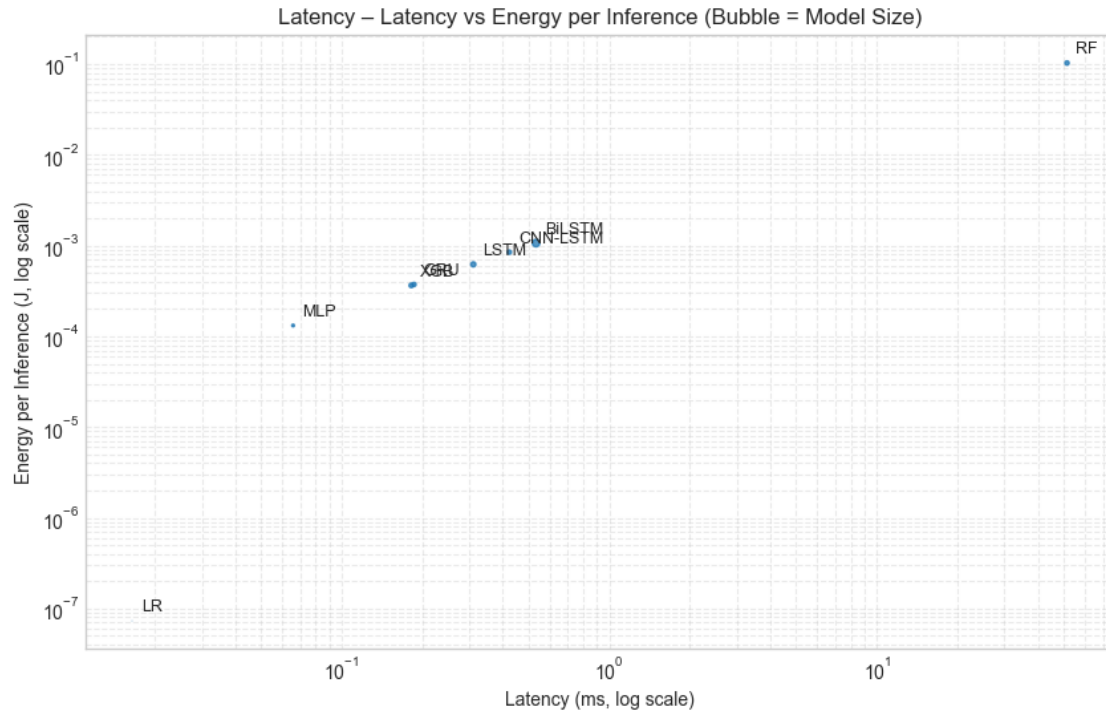
```
[40]: generate_edge_comparison_plots(metric="Latency",showImage=True)
```





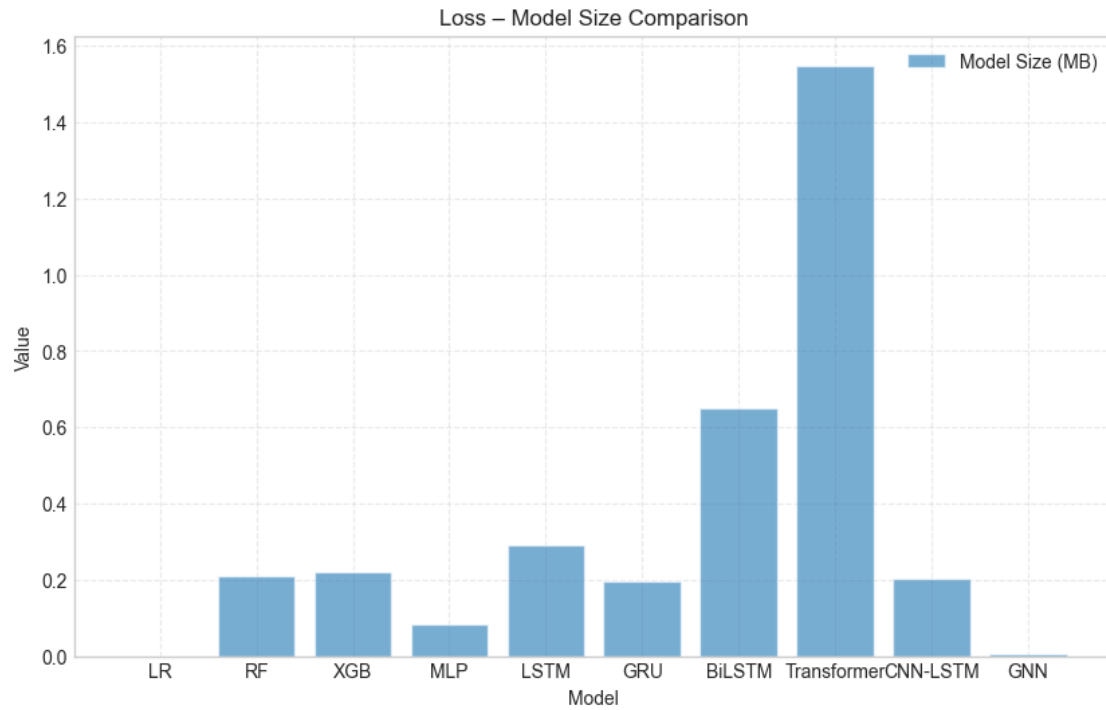


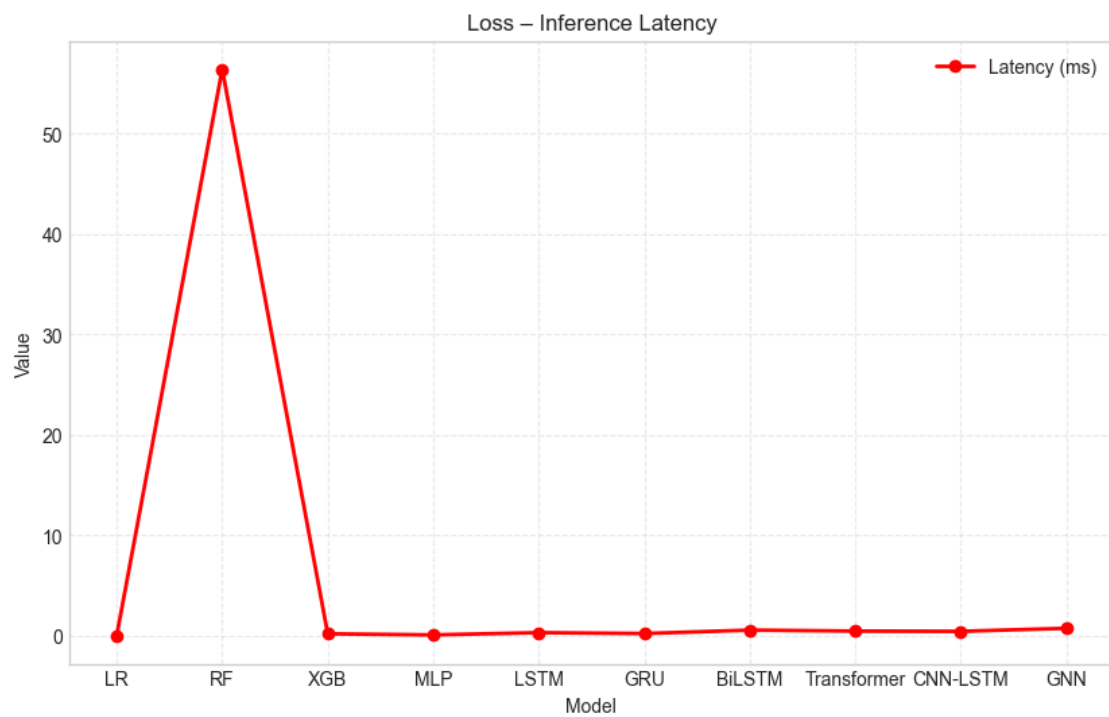
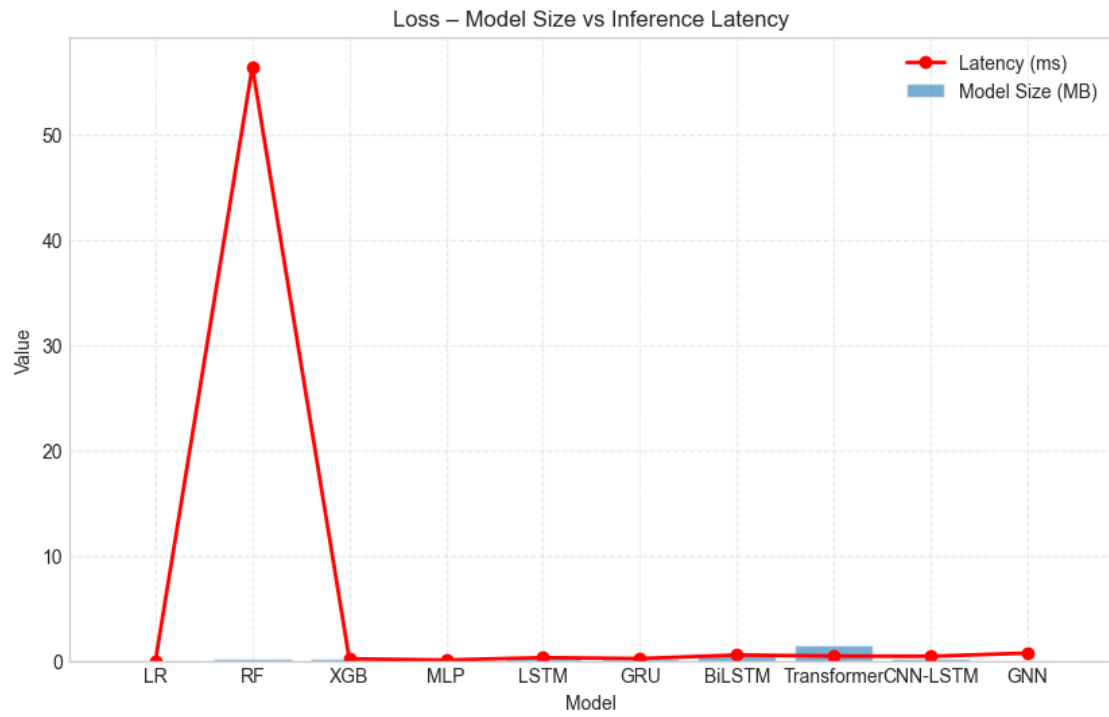


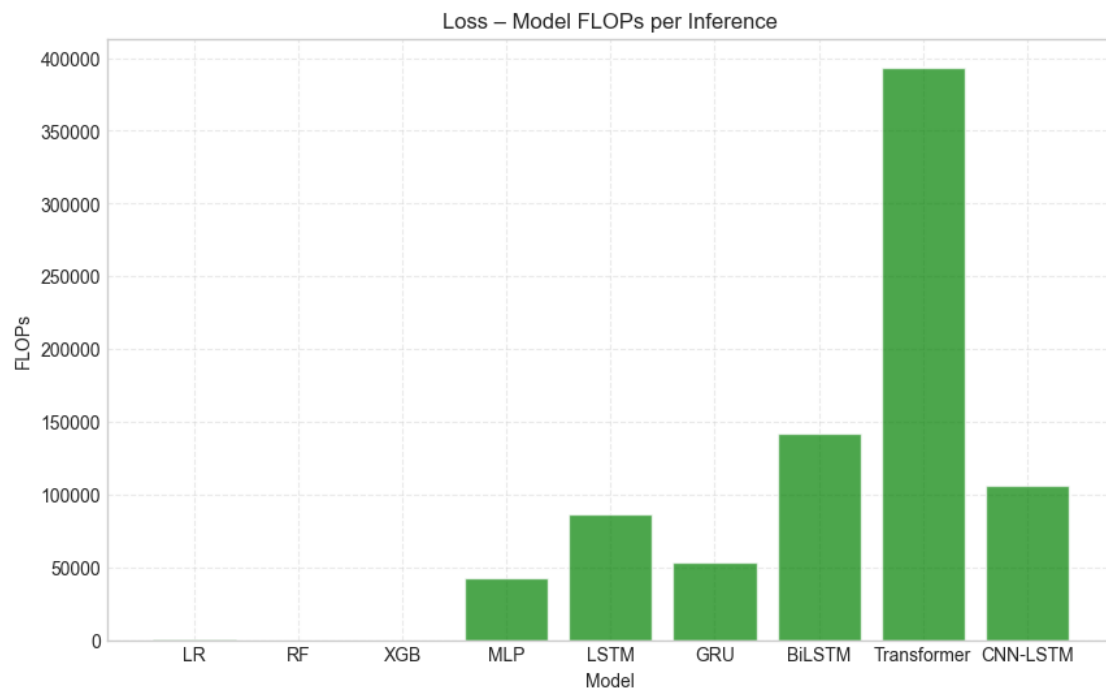
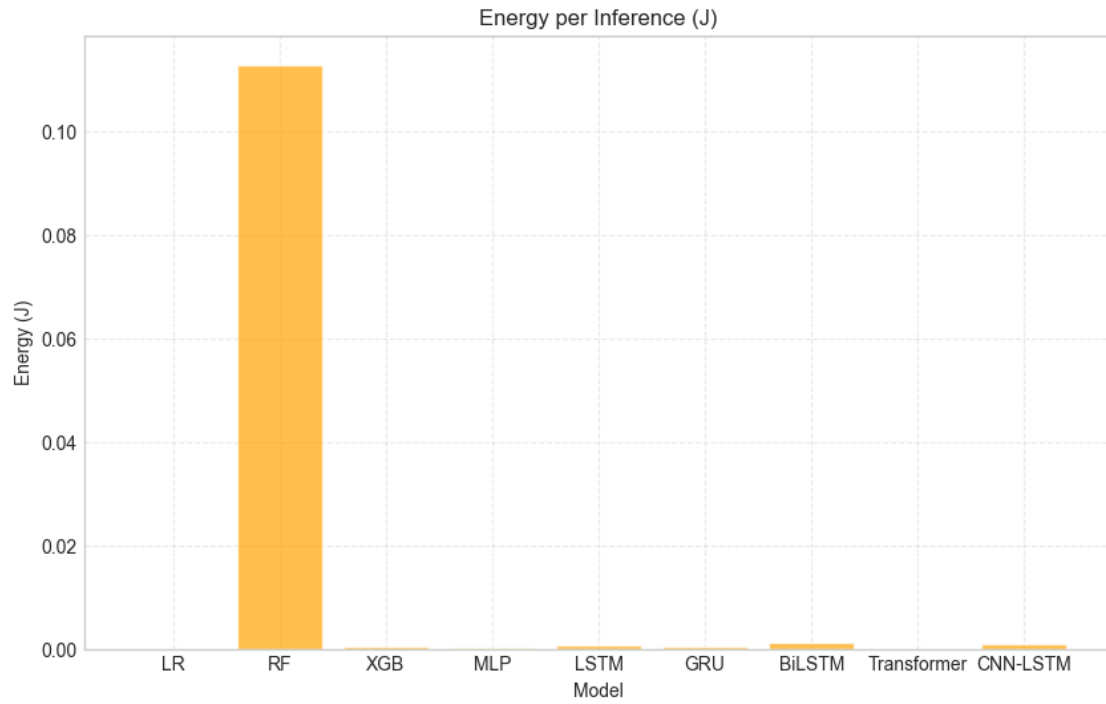


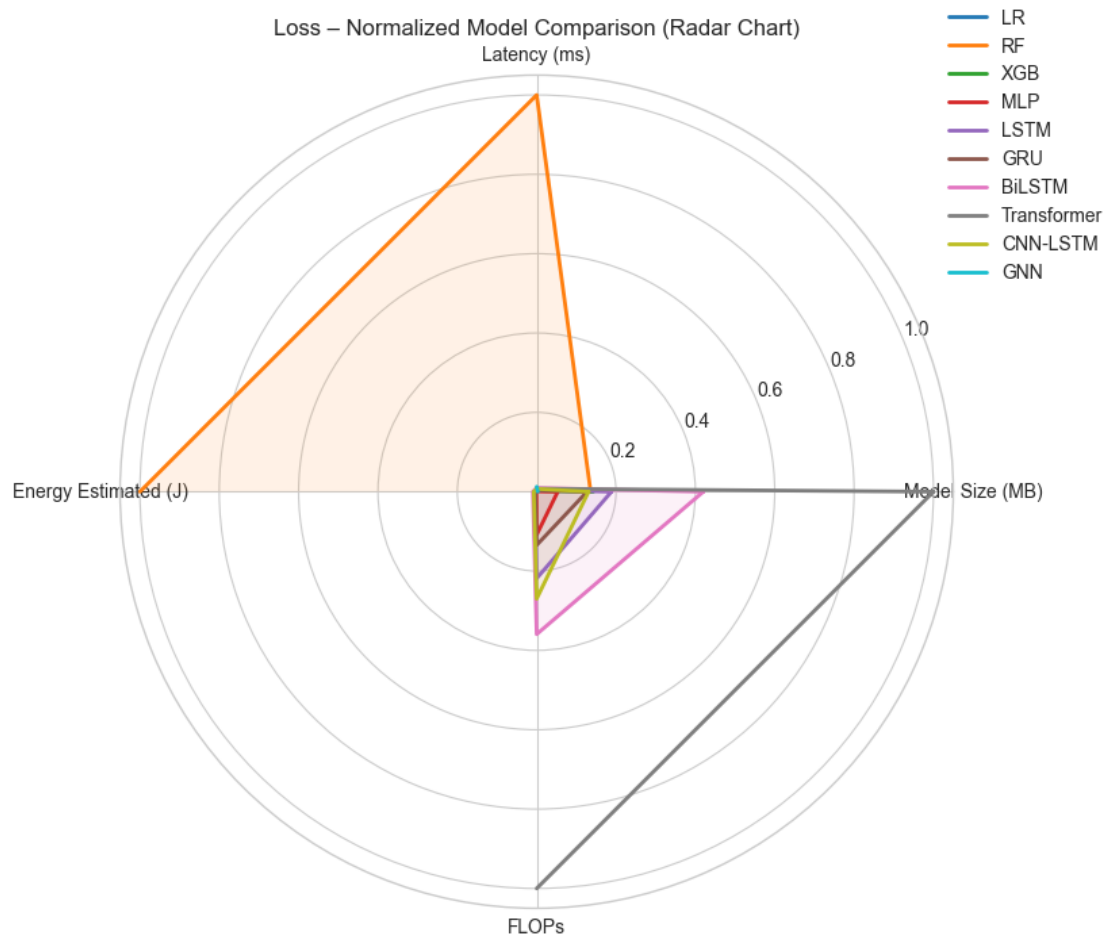
All Latency edge comparison plots generated successfully.

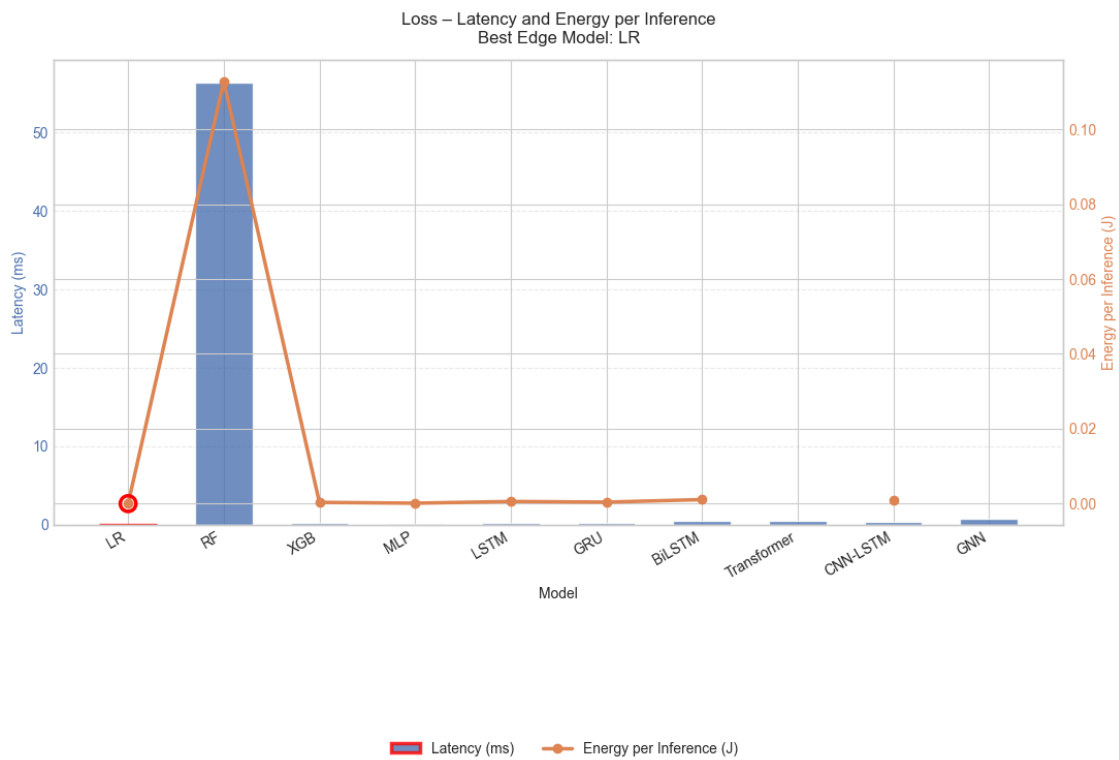
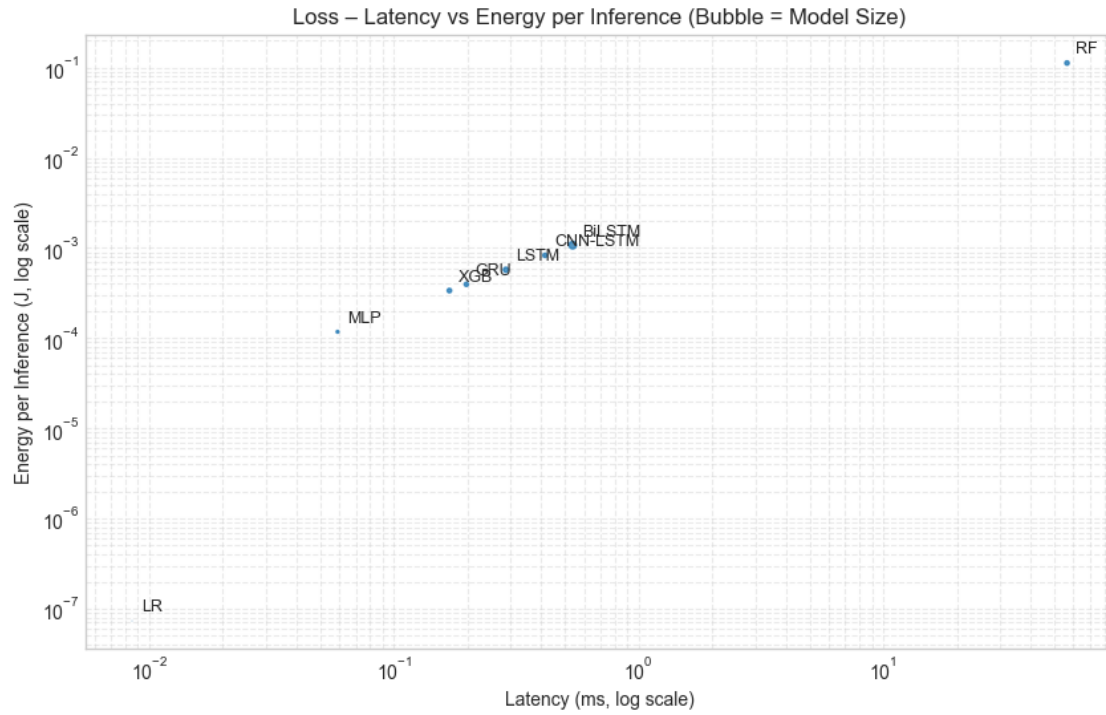
```
[41]: generate_edge_comparison_plots(metric="Loss",showImage=True)
```





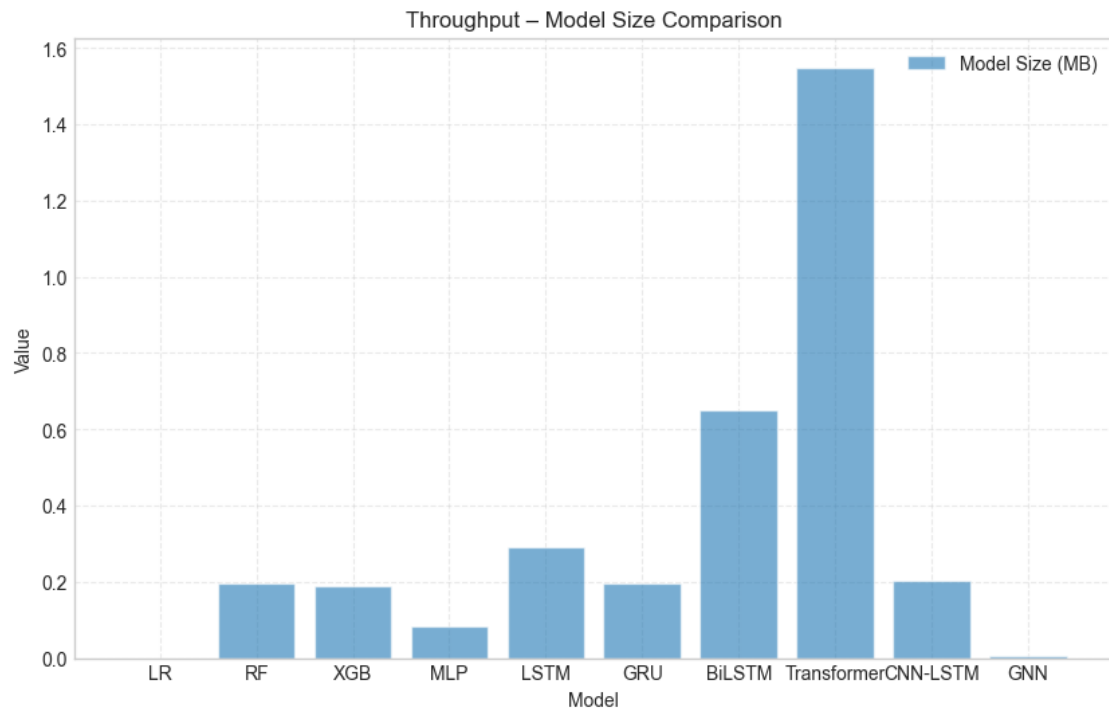


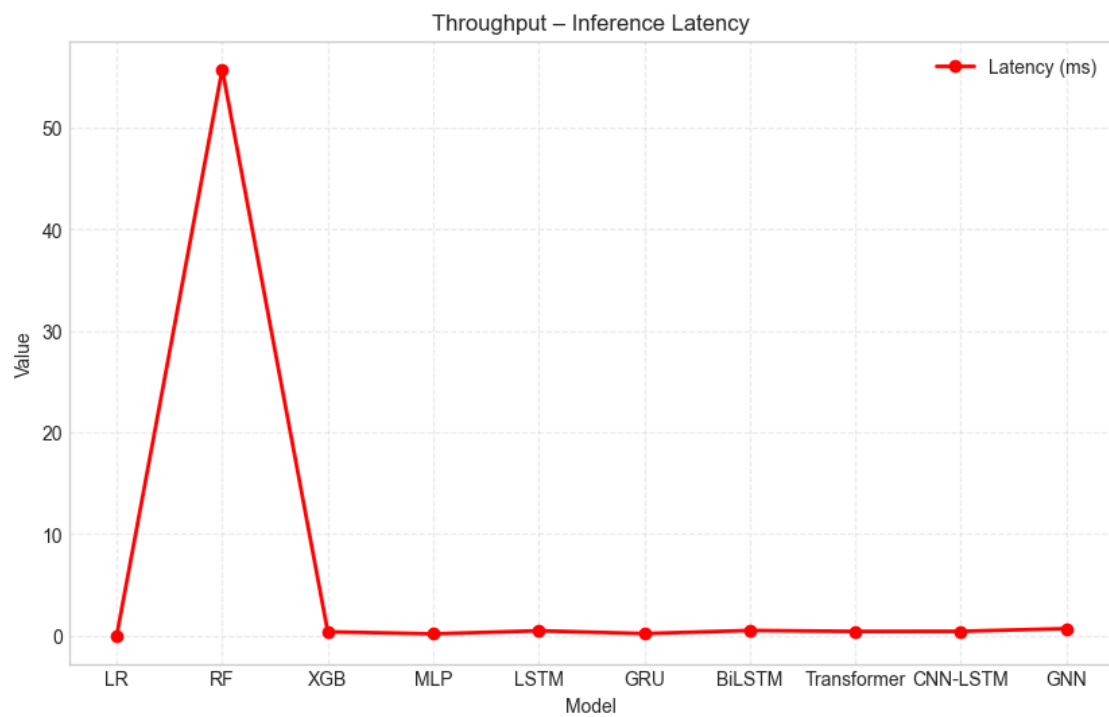
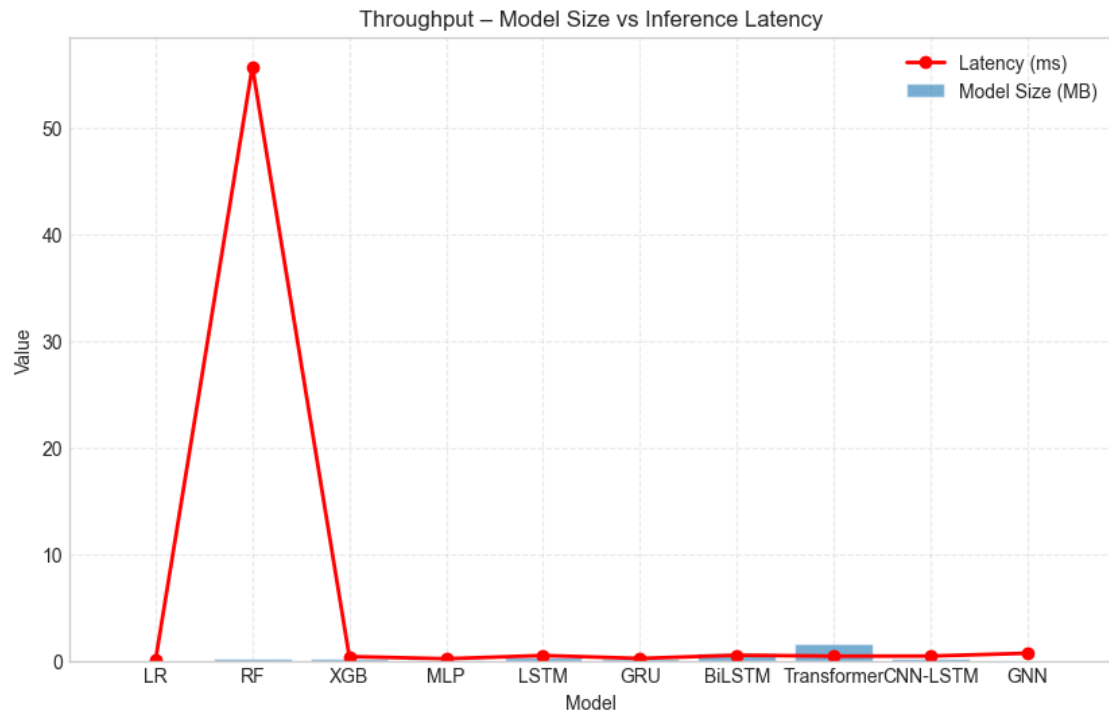


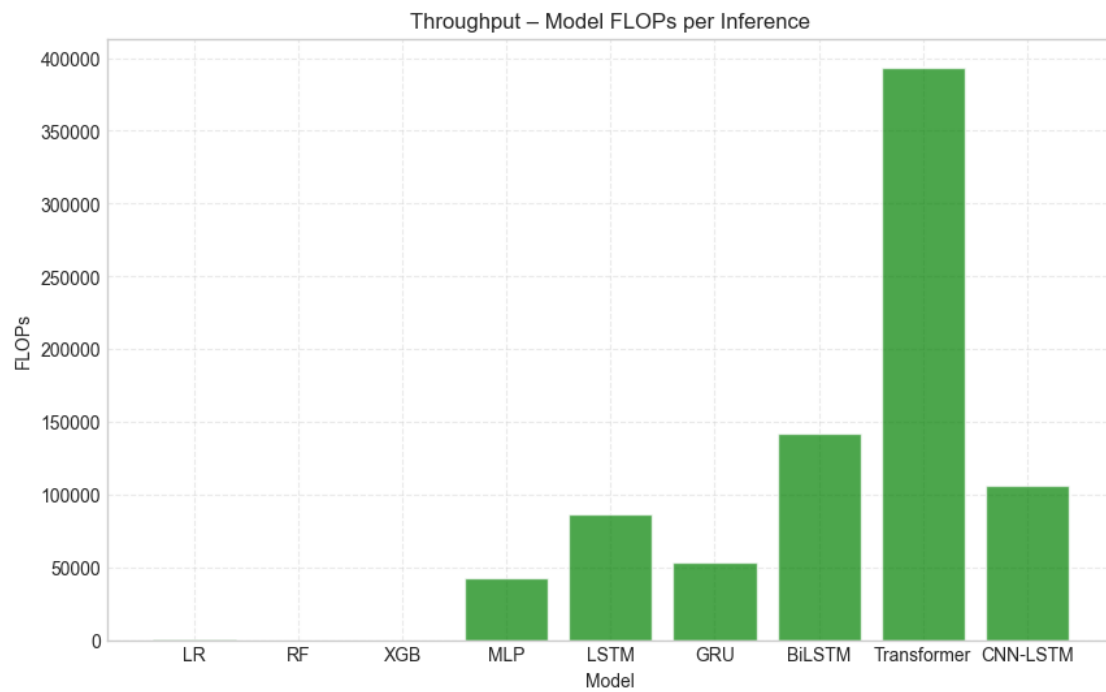
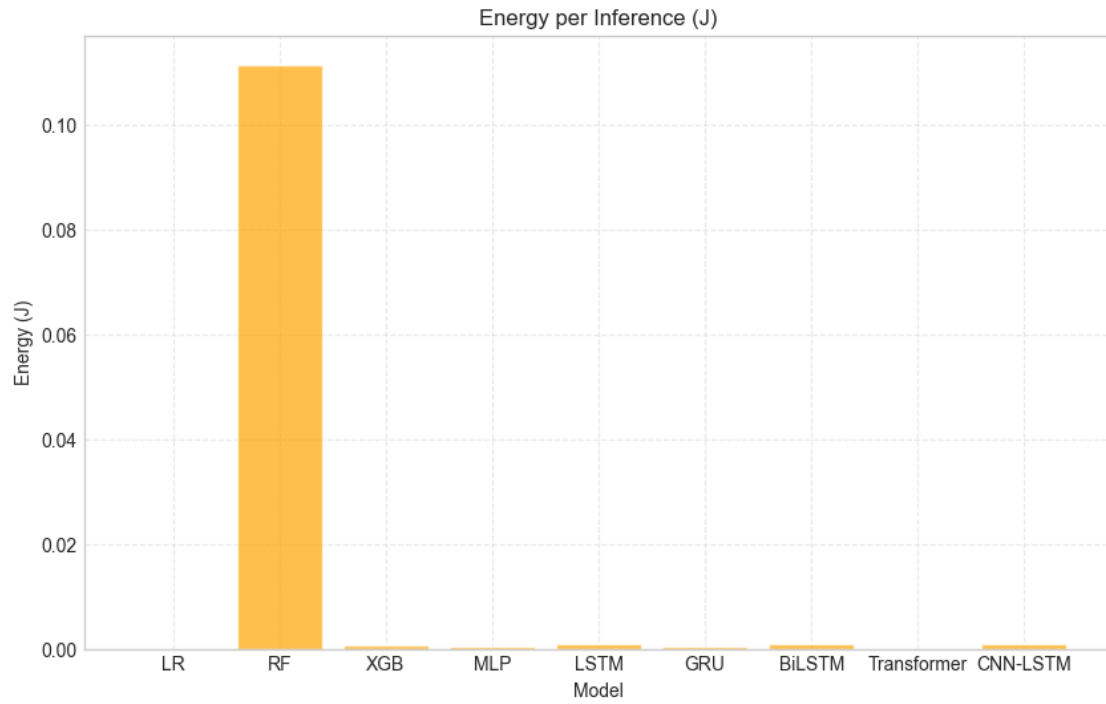


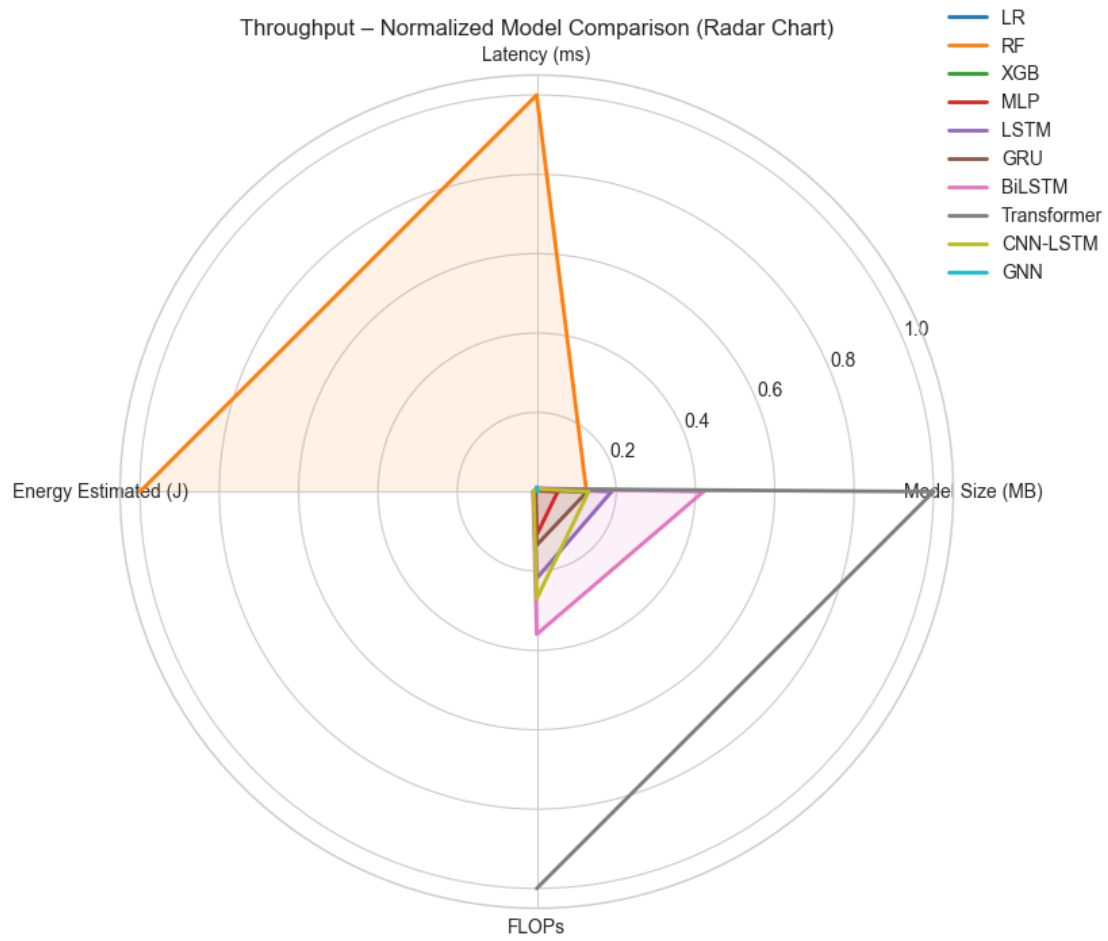
All Loss edge comparison plots generated successfully.

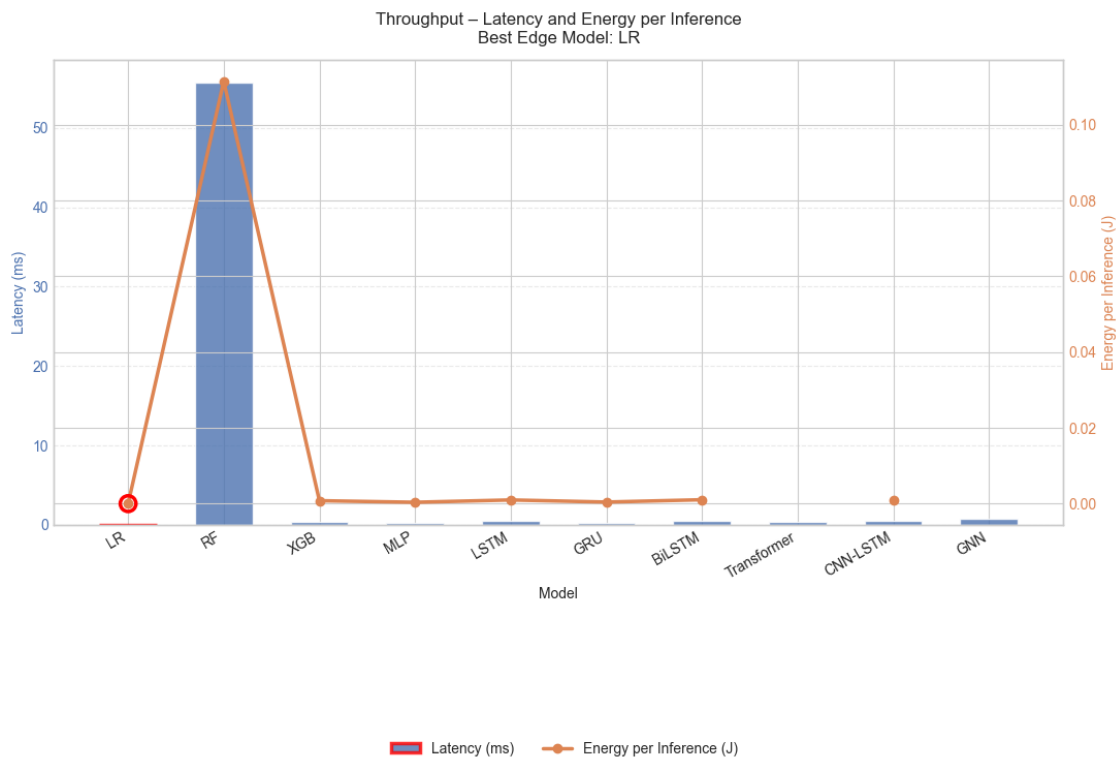
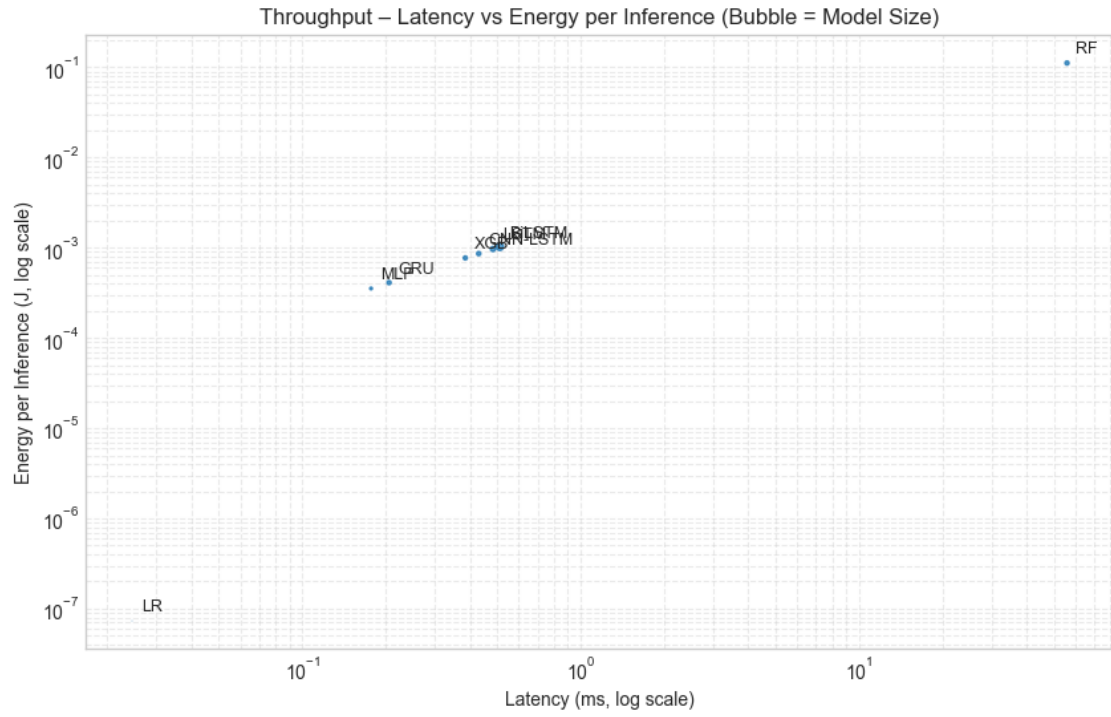
```
[42]: generate_edge_comparison_plots(showImage=True)
```











All Throughput edge comparison plots generated successfully.

```
[43]: from pathlib import Path

def find_single_csv(base_dir: Path, keyword: str):
    """
    Find exactly one CSV in base_dir containing keyword.
    """
    matches = list(base_dir.glob(f"*{keyword}*.csv"))

    if len(matches) == 0:
        raise FileNotFoundError(
            f"No CSV found in {base_dir} containing '{keyword}'"
        )
    if len(matches) > 1:
        raise ValueError(
            f"Multiple CSVs found in {base_dir} containing '{keyword}':\n"
            + "\n".join(str(p) for p in matches)
        )
    return matches[0]
```

```
[44]: import pandas as pd
import numpy as np

def compute_accuracy_from_predictions(pred_csv, error_type="rmse"):
    df = pd.read_csv(pred_csv)

    required = {"Model", "y_true", "y_pred"}
    if not required.issubset(df.columns):
        raise ValueError(f"{pred_csv} must contain {required}")

    rows = []
    for model, g in df.groupby("Model"):
        y_true = g["y_true"].values
        y_pred = g["y_pred"].values

        if error_type == "rmse":
            error = np.sqrt(np.mean((y_true - y_pred) ** 2))
        elif error_type == "mae":
            error = np.mean(np.abs(y_true - y_pred))
        else:
            raise ValueError("error_type must be 'rmse' or 'mae'")

        rows.append({
```

```

        "Model": model,
        "AccuracyError": error,
    })

    return pd.DataFrame(rows)

```

```

[45]: import pandas as pd
import numpy as np

def compute_accuracy_from_single_pred_csv(
    csv_path,
    error_type="rmse",
):
    """
    Compute accuracy from one model's prediction CSV.
    """
    df = pd.read_csv(csv_path)

    required = {"y_true", "y_pred"}
    if not required.issubset(df.columns):
        raise ValueError(
            f"{csv_path} must contain columns {required}"
        )

    y_true = df["y_true"].values
    y_pred = df["y_pred"].values

    if error_type == "rmse":
        return np.sqrt(np.mean((y_true - y_pred) ** 2))
    elif error_type == "mae":
        return np.mean(np.abs(y_true - y_pred))
    else:
        raise ValueError("error_type must be 'rmse' or 'mae'")

```

```

[46]: def normalize_model_key(model_name: str) -> str:
    """
    Convert model display name to filename-safe key.
    """
    return (
        model_name
        .lower()
        .replace("-", "_")
        .replace(" ", "")
    )

```

```

[47]: METRIC_SUFFIX = {
        "Throughput": "throughput",
        "Latency": "latency",
        "Loss": "loss",
    }

from pathlib import Path

from pathlib import Path
import pandas as pd

def generate_accuracy_vs_edge_efficiency_auto(
    metric="Latency",
    split="test",
    error_type="rmse",
):
    """
    Generate accuracy vs edge efficiency CSV using:
    - Edge metrics from run_all_edge_metrics()
    - Per-model prediction CSVs from ../outputs/models/predictions/

    Expected prediction CSV format:
    {model_key}_{metric}_{split}_predictions.csv
    """

    metric_key = METRIC_SUFFIX[metric]

    edge_dir = Path("../outputs/edge_analysis")
    pred_dir = Path("../outputs/models/predictions")

    edge_csv = edge_dir / f"{metric_key}_edge_metrics.csv"
    if not edge_csv.exists():
        raise FileNotFoundError(f"Missing edge CSV: {edge_csv}")

    edge_df = pd.read_csv(edge_csv)
    if "Unnamed: 0" in edge_df.columns:
        edge_df = edge_df.rename(columns={"Unnamed: 0": "Model"})

    rows = []

    for model_name in edge_df["Model"].unique():

        model_key = normalize_model_key(model_name)

        pred_csv = (
            pred_dir /
            f"{model_key}_{metric}_{split}_predictions.csv"

```

```

    )

    if not pred_csv.exists():
        raise FileNotFoundError(
            f"Missing prediction CSV for model '{model_name}'.\n"
            f"Expected: {pred_csv}\n"
            f"Available files: {[p.name for p in pred_dir.
↪glob(f'{model_key}_*_{split}_predictions.csv')]]}"
        )

    acc_error = compute_accuracy_from_single_pred_csv(
        pred_csv,
        error_type=error_type
    )

    edge_row = edge_df[edge_df["Model"] == model_name].iloc[0]

    rows.append({
        "Model": model_name,
        "AccuracyError": acc_error,
        "Latency_ms": edge_row["Latency (ms)"],
        "ModelSize_MB": edge_row["Model Size (MB)"],
        "FLOPs": edge_row["FLOPs"],
        "Energy_J": edge_row["Energy Estimated (J)"],
    })

    out_df = pd.DataFrame(rows)

    out_path = (
        edge_dir /
        f"{metric_key}_{split}_accuracy_edge_tradeoff.csv"
    )
    out_df.to_csv(out_path, index=False)

    print(f"Saved accuracy-edge trade-off CSV → {out_path}")
    return out_df

```

```

[48]: df_tradeoff = generate_accuracy_vs_edge_efficiency_auto(
    metric="Throughput",
    split="test",
    error_type="rmse",
)

# print(df_tradeoff)

```

Saved accuracy-edge trade-off CSV →
 ../outputs/edge_analysis/throughput_test_accuracy_edge_tradeoff.csv

```
[49]: df_tradeoff = generate_accuracy_vs_edge_efficiency_auto(
        metric="Latency",
        split="test",
        error_type="rmse",
    )
```

Saved accuracy-edge trade-off CSV →
 ../outputs/edge_analysis/latency_test_accuracy_edge_tradeoff.csv

```
[50]: df_tradeoff = generate_accuracy_vs_edge_efficiency_auto(
        metric="Loss",
        split="test",
        error_type="rmse",
    )
```

Saved accuracy-edge trade-off CSV →
 ../outputs/edge_analysis/loss_test_accuracy_edge_tradeoff.csv

```
[51]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

def plot_accuracy_vs_edge_efficiency_grouped_bar(
    metric="Latency",
    split="test",
    accuracy_col="AccuracyError",
    latency_col="Latency_ms",
    model_col="Model",
    edge_dir="../outputs/edge_analysis",
    showImage=False,
):
    """
    Grouped bar chart for Accuracy vs Edge Efficiency.

    Reads:
        ../outputs/edge_analysis/{metric}_{split}_accuracy_edge_tradeoff.csv

    Saves:
        ../outputs/edge_analysis/plots/{metric}/
        ↪compare_{metric}_{split}_edge_grouped_bar.png
    """

    metric_key = metric.lower()

    edge_dir = Path(edge_dir)
    plot_dir = edge_dir / "plots" / metric
```

```

plot_dir.mkdir(parents=True, exist_ok=True)

csv_path = edge_dir / f"{metric_key}_{split}_accuracy_edge_tradeoff.csv"
if not csv_path.exists():
    raise FileNotFoundError(f"Trade-off CSV not found: {csv_path}")

df = pd.read_csv(csv_path)

# Sanity check
for c in [model_col, accuracy_col, latency_col]:
    if c not in df.columns:
        raise ValueError(
            f"Missing column '{c}' in {csv_path.name}. "
            f"Available: {list(df.columns)}"
        )

models = df[model_col].tolist()
x = np.arange(len(models))
width = 0.35

plt.figure(figsize=(10, 5))

plt.bar(
    x - width / 2,
    df[accuracy_col],
    width,
    label="Prediction Error (RMSE)"
)

plt.bar(
    x + width / 2,
    df[latency_col],
    width,
    label="Inference Latency (ms)"
)

plt.xlabel("Model")
plt.ylabel("Value")
plt.title(
    f"Accuracy vs Edge Efficiency ({metric}, {split} set)"
)

plt.xticks(x, models, rotation=30, ha="right")
plt.legend()
plt.grid(axis="y", linestyle="--", alpha=0.4)

plt.tight_layout()

```

```

out_path = (
    plot_dir /
        f"compare_{metric_key}_{split}_edge_grouped_bar.png"
)
plt.savefig(out_path, dpi=300)
if showImage:
    plt.show()
plt.close()

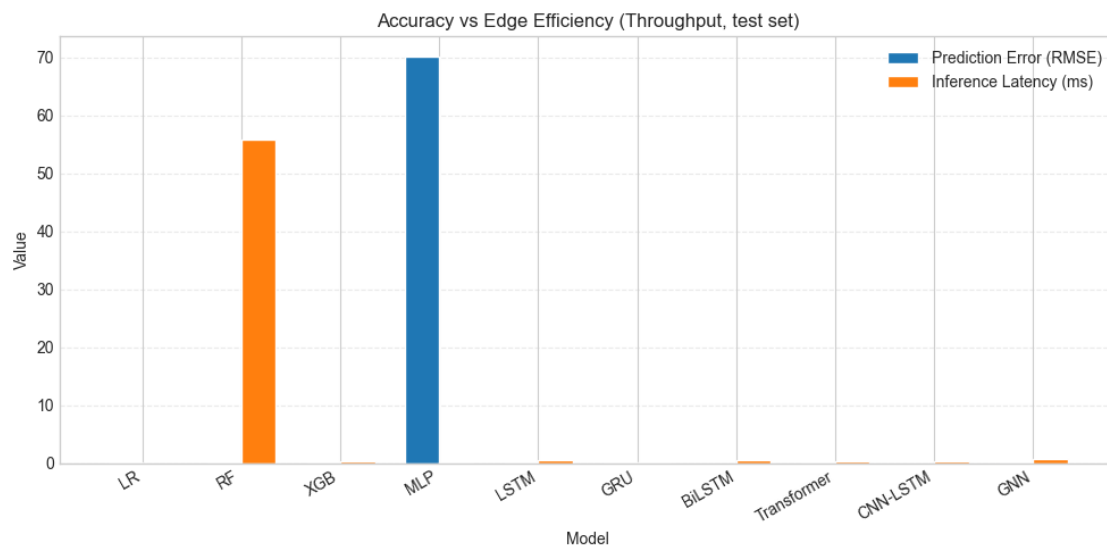
print(f"Saved grouped bar plot → {out_path}")

```

```

[52]: plot_accuracy_vs_edge_efficiency_grouped_bar(
    metric="Throughput",
    split="test",
    showImage=True
)

```

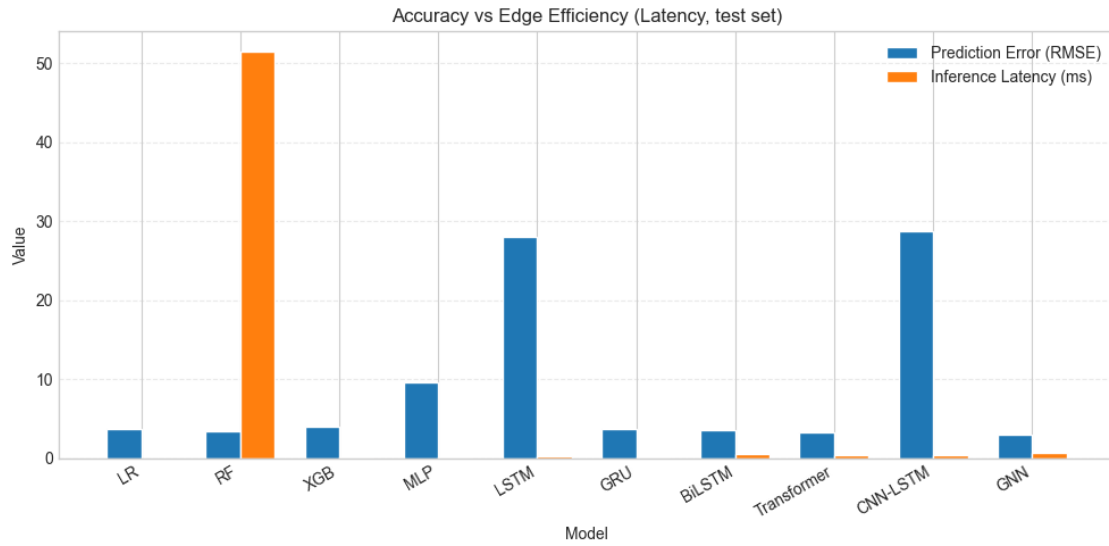


Saved grouped bar plot → ../outputs/edge_analysis/plots/Throughput/compare_throughput_test_edge_grouped_bar.png

```

[53]: plot_accuracy_vs_edge_efficiency_grouped_bar(
    metric="Latency",
    split="test",
    showImage=True
)

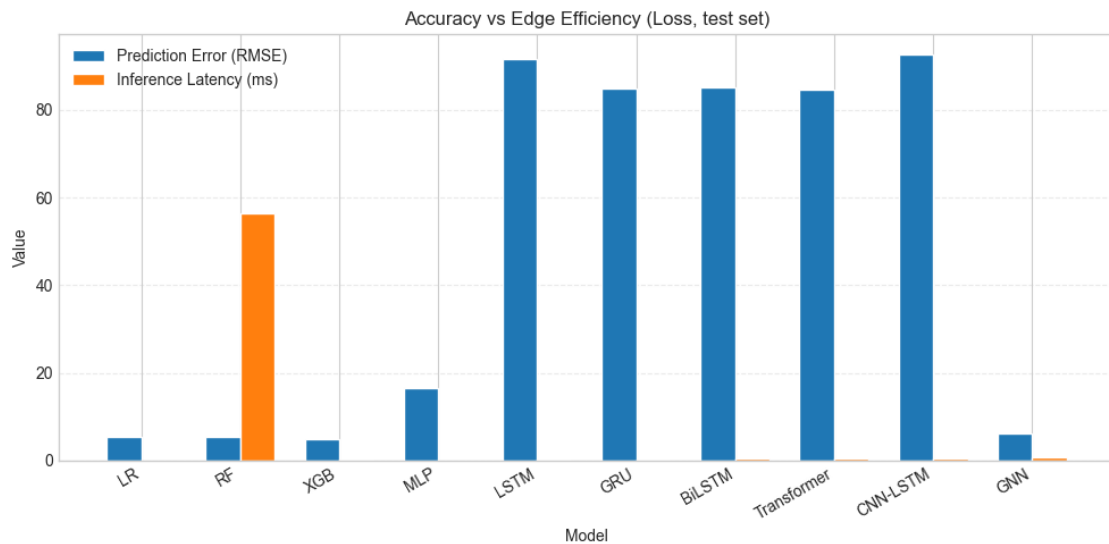
```



Saved grouped bar plot →

../outputs/edge_analysis/plots/Latency/compare_latency_test_edge_grouped_bar.png

```
[54]: plot_accuracy_vs_edge_efficiency_grouped_bar(
    metric="Loss",
    split="test",
    showImage=True
)
```



Saved grouped bar plot →

../outputs/edge_analysis/plots/Loss/compare_loss_test_edge_grouped_bar.png

```

[55]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

def plot_accuracy_bar_latency_line(
    metric="Latency",
    split="test",
    accuracy_col="AccuracyError",
    latency_col="Latency_ms",
    model_col="Model",
    edge_dir="../outputs/edge_analysis",
    showImage=False,
):
    """
    Plot prediction error as BAR and inference latency as LINE.

    Reads:
        ../outputs/edge_analysis/{metric}_{split}_accuracy_edge_tradeoff.csv

    Saves:
        ../outputs/edge_analysis/plots/{metric}/
        ↪compare_{metric}_{split}_edge_bar_line.png
    """

    metric_key = metric.lower()

    edge_dir = Path(edge_dir)
    plot_dir = edge_dir / "plots" / metric
    plot_dir.mkdir(parents=True, exist_ok=True)

    csv_path = edge_dir / f"{metric_key}_{split}_accuracy_edge_tradeoff.csv"
    if not csv_path.exists():
        raise FileNotFoundError(f"Trade-off CSV not found: {csv_path}")

    df = pd.read_csv(csv_path)

    # Sanity check
    for c in [model_col, accuracy_col, latency_col]:
        if c not in df.columns:
            raise ValueError(
                f"Missing column '{c}' in {csv_path.name}. "
                f"Available: {list(df.columns)}"
            )

    models = df[model_col].tolist()

```

```

x = np.arange(len(models))

fig, ax1 = plt.subplots(figsize=(10, 5))

# -----
# Bar: Prediction Error
# -----
bars = ax1.bar(
    x,
    df[accuracy_col],
    width=0.6,
    alpha=0.8,
    label="Prediction Error (RMSE)"
)

ax1.set_xlabel("Model")
ax1.set_ylabel("Prediction Error (RMSE)")
ax1.set_xticks(x)
ax1.set_xticklabels(models, rotation=30, ha="right")
ax1.grid(axis="y", linestyle="--", alpha=0.4)

# -----
# Line: Inference Latency
# -----
ax2 = ax1.twinx()
ax2.plot(
    x,
    df[latency_col],
    color="red",
    marker="o",
    linewidth=2,
    label="Inference Latency (ms)"
)
ax2.set_ylabel("Inference Latency (ms)")

# -----
# Title & Legend
# -----
plt.title(
    f"Prediction Accuracy vs Edge Efficiency ({metric}, {split} set)"
)

# Combined legend
lines_labels = (
    ax1.get_legend_handles_labels()[0]
    + ax2.get_legend_handles_labels()[0],
    ax1.get_legend_handles_labels()[1]

```

```

    + ax2.get_legend_handles_labels()[1]
)
ax1.legend(*lines_labels, loc="upper left")

plt.tight_layout()

out_path = (
    plot_dir /
    f"compare_{metric_key}_{split}_edge_bar_line.png"
)
plt.savefig(out_path, dpi=300)
if showImage:
    plt.show()
plt.close()

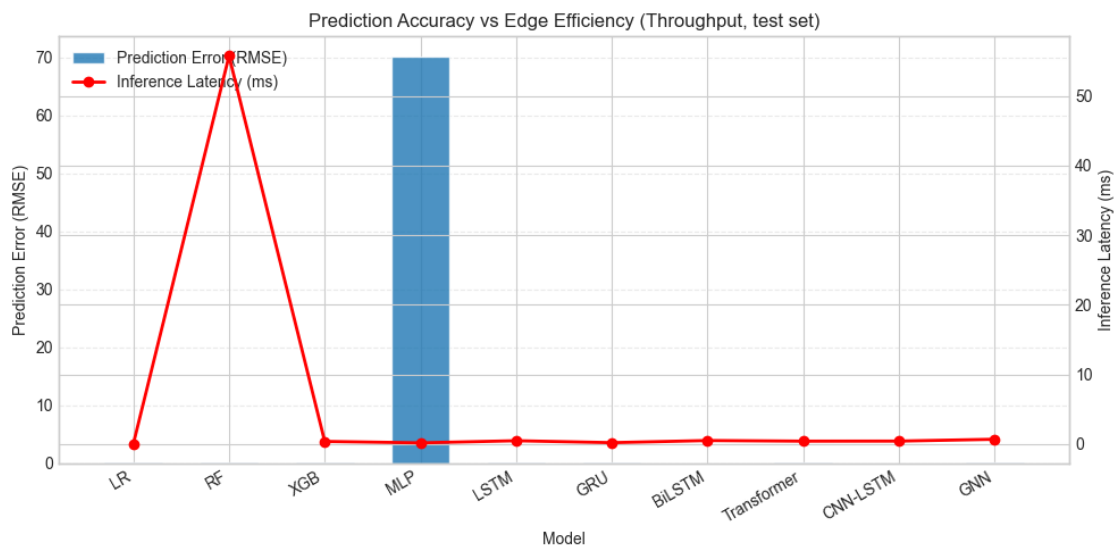
print(f"Saved bar-line plot → {out_path}")

```

```

[56]: plot_accuracy_bar_latency_line(
    metric="Throughput",
    split="test",
    showImage=True
)

```



Saved bar-line plot → ../outputs/edge_analysis/plots/Throughput/compare_throughput_test_edge_bar_line.png

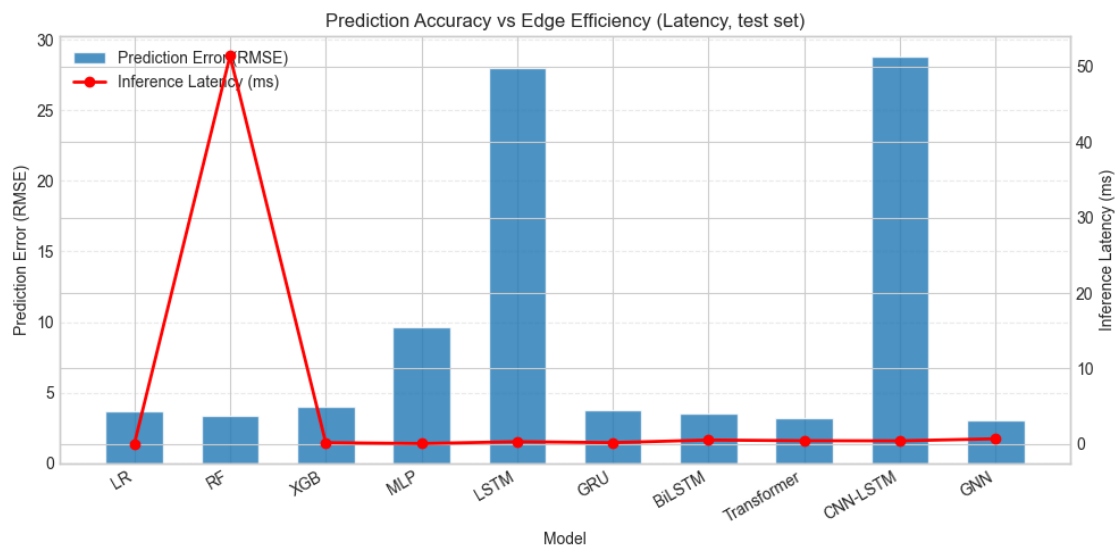
```

[57]: plot_accuracy_bar_latency_line(
    metric="Latency",
    split="test",

```

```
showImage=True
```

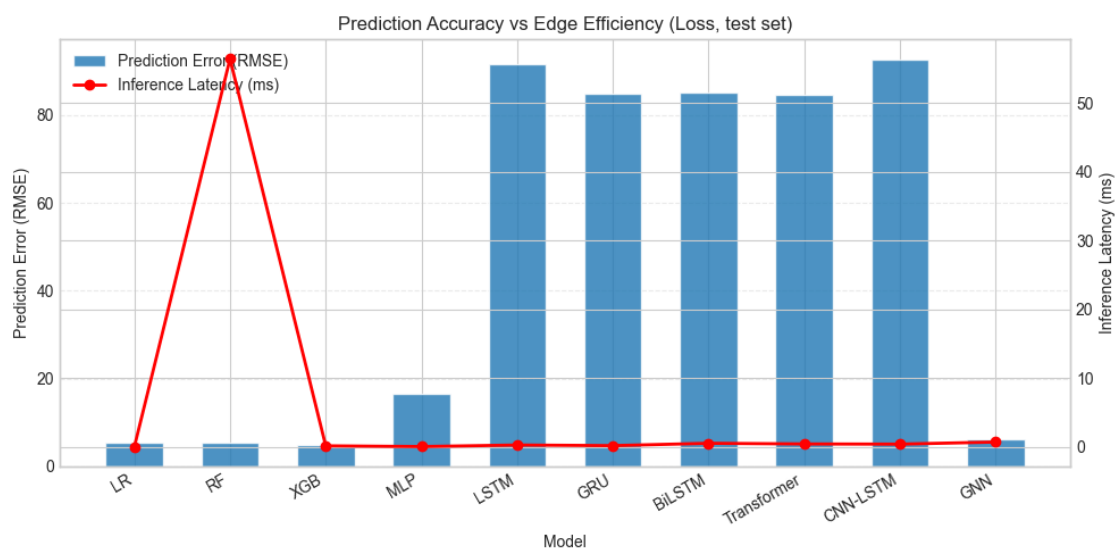
```
)
```



Saved bar-line plot →

../outputs/edge_analysis/plots/Latency/compare_latency_test_edge_bar_line.png

```
[58]: plot_accuracy_bar_latency_line(  
    metric="Loss",  
    split="test",  
    showImage=True  
)
```



Saved bar-line plot →

../outputs/edge_analysis/plots/Loss/compare_loss_test_edge_bar_line.png

[]:

